
Learning with Local Search MCMC Layers

Germain Vivier-Ardisson
Google DeepMind & ENPC
Paris, France
gvivier@google.com

Axel Parmentier
ENPC
Marne-la-Vallée, France
axel.parmentier@enpc.fr

Mathieu Blondel
Google DeepMind
Paris, France
mblondel@google.com

Abstract

Integrating combinatorial optimization layers into neural networks has recently attracted significant research interest. However, many existing approaches lack theoretical guarantees or fail to perform adequately when relying on inexact solvers. This is a critical limitation, as many operations research problems are NP-hard, often necessitating the use of neighborhood-based local search heuristics. These heuristics iteratively generate and evaluate candidate solutions based on an acceptance rule. In this paper, we introduce a theoretically-principled approach for learning with such inexact combinatorial solvers. Inspired by the connection between simulated annealing and Metropolis-Hastings, we propose to transform problem-specific neighborhood systems used in local search heuristics into proposal distributions, implementing MCMC on the combinatorial space of feasible solutions. This allows us to construct differentiable combinatorial layers and associated loss functions. Replacing an exact solver by a local search strongly reduces the computational burden of learning on many applications. We demonstrate our approach on a large-scale dynamic vehicle routing problem with time windows.

1 Introduction

Models that combine neural networks and combinatorial optimization have recently attracted significant attention [14, 39, 8, 6, 52, 5, 34, 45, 7]. Such models enable the transformation of learned continuous latent representations into structured discrete outputs that satisfy complex constraints. They enrich combinatorial optimization algorithms by providing them with context-dependent features, making decisions more resilient to uncertainty. An important subset of this line of research involves integrating, within a neural network, a linear programming layer of the form:

$$\theta \mapsto \operatorname{argmax}_{y \in \mathcal{Y}} \langle \theta, y \rangle \subseteq \operatorname{argmax}_{y \in \operatorname{conv}(\mathcal{Y})} \langle \theta, y \rangle, \quad (1)$$

where $\mathcal{Y}$ is a finite set of feasible outputs. In the graphical model and structured prediction literature, this is often called the maximum a posteriori (MAP) problem [53]. The main challenge in using such layers lies in their end-to-end model training. Indeed, as piecewise-constant, discontinuous functions, such layers break the differentiable programming computational graph, and prevent one from backpropagating meaningful gradients from the final output of the model to its parameters.

Many approaches have been proposed to derive relaxations and loss functions for this setting; see Section 2 for a detailed review. As outlined in Table 1, these approaches often differ in the oracle (solver) they assume access to. Some rely on an oracle for solving a continuous relaxation of the above linear program, such as a quadratic or entropy-regularized program. They typically perform a single oracle call per data point. Some other approaches assume access to an oracle for solving the original linear program (i.e., a MAP oracle), but perform multiple oracle calls, for smoothing reasons. Theoretical guarantees for these approaches usually assume an oracle returning exact solutions.

Table 1: The proposed approach leverages the neighborhood systems used by local search heuristics (inexact solvers) to obtain a differentiable combinatorial layer when usual oracles are not available.

	Regularization	Oracle	Approach
Differentiable DP (2009, 2018)	Entropy	Exact marginal	DP
SparseMAP (2018)	Quadratic	Exact MAP	Frank-Wolfe
Barrier FW (2015)	TRW Entropy	Exact MAP	Frank-Wolfe
IntOpt (2020)	Log barrier	Interior point solver	Primal-Dual
Perturbed optimizers (2020)	Implicit via noise	Exact MAP	Monte-Carlo
Blackbox solvers (2020)	None	Exact MAP	Interpolation
Contrastive divergences (2000)	Entropy	Gibbs / Langevin sampler	MCMC
Proposed	Entropy	Local search	MCMC

Unfortunately, many problems in operations research are NP-hard in nature, making access to an exact oracle difficult. In contrast, operations research applications often rely on local search heuristics, such as simulated annealing. These heuristics iteratively generate a neighbor of the current solution, and either accept it or reject it based on an acceptance rule. The aim of this work is to provide a theoretically-principled approach for learning with such inexact combinatorial solvers. This is crucial for exploiting widely-used advances in heuristics from the operations research literature as layers in neural networks.

To do so, we propose to leverage the links between neighborhood-based, local search heuristics used to approximately solve combinatorial problems, and Markov chain Monte-Carlo (MCMC) methods used to perform approximate marginal inference in graphical models. These lines of research have evolved separately and the links between them have not been exploited for designing principled machine learning pipelines. We make the following contributions:

- We propose new differentiable MCMC layers in combinatorial spaces. To do so, we propose to transform problem-specific neighborhood systems that make local search heuristics efficient into proposal distributions to implement MCMC on the combinatorial space of feasible solutions.
- We show that there exist Fenchel-Young losses [8] (even with a single MCMC iteration) whose stochastic gradients are given by the proposed layer, leading to principled algorithms for both the supervised and unsupervised learning settings, for which we provide a convergence analysis.
- Using our proposed MCMC layer strongly alleviates the computational bottleneck, even more so with few MCMC iterations. This allows us to handle larger problem instances in the training set, leading to better generalization on large-scale applications [40], which is critical in operations research, as it reduces marginal costs [41].
- We demonstrate our approach on the EURO Meets NeurIPS 2022 challenge [27], a large-scale, ML-enriched dynamic vehicle routing problem with time windows (DVRPTW), which involves an intractable combinatorial optimization problem. In Appendix A, we also empirically validate the quality of the proposed gradient estimator through abundant experiments.

2 Background

2.1 Problem setup

In this paper, our goal is to learn models with an optimization layer of the form

$$\hat{\mathbf{y}}(\boldsymbol{\theta}) := \underset{\mathbf{y} \in \mathcal{Y}}{\operatorname{argmax}} \langle \boldsymbol{\theta}, \mathbf{y} \rangle + \varphi(\mathbf{y}), \quad (2)$$

where $\boldsymbol{\theta} \in \mathbb{R}^d$ and $\mathcal{Y} \subset \mathbb{R}^d$ is a finite but combinatorially-large set. Eq. (2) is a slight generalization of Eq. (1), where $\varphi : \mathcal{Y} \rightarrow \mathbb{R}$ can be used to incorporate a penalty on the solution. It may no longer be a continuous linear program, as φ could potentially be undefined on $\operatorname{conv}(\mathcal{Y})$. We focus on settings where this optimization problem is intractable and only heuristic algorithms are available to obtain an approximate solution. Our goal is to integrate NP-hard problems arising in operations research (e.g., routing, scheduling, network design), within a neural network. Unfortunately, many existing approaches simply do not work or lack formal guarantees when used with inexact solvers.

We distinguish between two settings. In the unsupervised setting, our goal will be to learn $\boldsymbol{\theta} \in \mathbb{R}^d$ from observations $\mathbf{y}_1, \dots, \mathbf{y}_N \in \mathbb{R}^d$. In the supervised setting, we will assume that $\boldsymbol{\theta} = g_W(\mathbf{x})$ and our goal will be to learn the parameters W from observation pairs $(\mathbf{x}_1, \mathbf{y}_1), \dots, (\mathbf{x}_N, \mathbf{y}_N)$.

2.2 Combinatorial optimization as a layer

Because the function in Eq. (1) is piecewise constant and discontinuous, a frequent strategy consists in introducing regularization in the problem so as to obtain a continuous relaxation. In some cases, we may have access to an oracle for directly solving the regularized problem. For instance, when the unregularized problem can be solved by dynamic programming, its entropic regularization can be computed using a change of semi-ring [31] or by algorithmic smoothing [35]. As another example, interior point solvers can be used to compute a logarithmic barrier regularized solution [33].

We focus on the setting where we only have access to a MAP oracle for solving the original unregularized problem 1, as tackled in several works, or more generally problem 2, as permitted by our work. Frank-Wolfe-like methods can be used to solve the regularized problem using only MAP oracle calls [39, 28]. Another strategy consists in injecting noise perturbations [6] in the oracle. This approach can be shown to be implicitly using regularization. In both cases, a Fenchel-Young loss can be associated, giving a principled way to learn with the optimization layer. However, formal guarantees only hold if the oracle used is exact, and in practice it is typically called multiple times during the forward pass. Our proposal enjoys guarantees even with inexact solvers and a single call.

Regarding differentiation, several strategies are possible. When the approach only needs to differentiate through a (regularized) max, as is the case of Fenchel-Young losses, we can use Danskin’s theorem [13]. When the approach needs to differentiate a (regularized) argmax, we can either use autodiff on the unrolled solver iterations or implicit differentiation [3, 1, 9]. Differently, Vlastelica et al. [52] propose to compute gradients via continuous interpolation of the solver.

2.3 Contrastive divergences

An alternative approach to learning in combinatorial spaces is to use energy-based models (EBMs) [30], which define a distribution over outputs via a parameterized energy function E_θ :

$$p_\theta(\mathbf{y}) \propto \exp(E_\theta(\mathbf{y})), \quad \text{with} \quad \nabla_\theta \log p_\theta(\mathbf{y}) = \nabla_\theta E_\theta(\mathbf{y}) - \mathbb{E}_{Y \sim p_\theta} [\nabla_\theta E_\theta(Y)].$$

Therefore, we can perform maximum likelihood estimation (MLE) if we can sample from p_θ , but this is hard both in continuous and combinatorial settings, due to its intractable normalization constant. Contrastive divergences [22, 11, 46] address this by using MCMC to obtain (biased) stochastic gradients. Originally developed for restricted Boltzmann machines with $\mathcal{Y} = \{0, 1\}^d$ and a Gibbs sampler, they have also been applied in continuous domains via Langevin dynamics [15, 16].

MCMC in discrete spaces. Contrastive divergences rely on MCMC to sample from the current model distribution. Unfortunately, designing an MCMC sampler is usually case by case, and MCMC on discrete domains has received comparatively less attention than continuous domains. Recent efforts adapt continuous techniques, such as Langevin dynamics [55, 47] or gradient-informed proposals [20, 42], to discrete settings. However, these works typically assume simple state space structure, like the hypercube or categorical codebooks, and do not handle complex constraints that are ubiquitous in operations research problems. Sun et al. [48] allow structured state spaces via relaxed constraints in the energy function, yet ignore these structures in the proposal supports. Notably, we emphasize that all these works focus on sampling, not on designing differentiable MCMC layers.

3 Local search based MCMC layers

In this section, we show how to design principled combinatorial layers without relying on exact MAP solvers, by transforming local search heuristics into MCMC algorithms.

3.1 From local search to MCMC

Local search and neighborhood systems. Local search heuristics [19] iteratively generate a neighbor $\mathbf{y}' \in \mathcal{N}(\mathbf{y}^{(k)})$ of the current solution $\mathbf{y}^{(k)}$, and either accept it or reject it based on an acceptance rule, that depends on the objective function, $\mathbf{y}^{(k)}$ and $\mathbf{y}'$. In this context, a neighborhood system $\mathcal{N}$ defines, for each feasible solution $\mathbf{y} \in \mathcal{Y}$, a set of neighbors $\mathcal{N}(\mathbf{y}) \subseteq \mathcal{Y}$. Neighborhoods are problem-specific, and must respect the structure of the problem, i.e., must maintain solution feasibility. They are typically defined implicitly via a set of allowed *moves* from $\mathbf{y}$. For instance, Table 2 lists example moves for a vehicle routing problem.

Algorithm 1 SA / MH as a layer

Inputs: $\theta \in \mathbb{R}^d, \mathbf{y}^{(0)} \in \mathcal{Y}, (t_k), K \in \mathbb{N}, \mathcal{N}, q$
for $k = 0 : K$ **do**
 Sample a neighbor in $\mathcal{N}(\mathbf{y}^{(k)})$:
 $\mathbf{y}' \sim q(\mathbf{y}^{(k)}, \cdot)$
 $\alpha(\mathbf{y}^{(k)}, \mathbf{y}') \leftarrow 1$ (SA) or
 $\alpha(\mathbf{y}^{(k)}, \mathbf{y}') \leftarrow \frac{q(\mathbf{y}', \mathbf{y}^{(k)})}{q(\mathbf{y}^{(k)}, \mathbf{y}')} \text{ (MH)}$
 $U \sim \mathcal{U}([0, 1])$
 $\Delta^{(k)} \leftarrow \langle \theta, \mathbf{y}' \rangle + \varphi(\mathbf{y}') - \langle \theta, \mathbf{y}^{(k)} \rangle - \varphi(\mathbf{y}^{(k)})$
 $p^{(k)} \leftarrow \alpha(\mathbf{y}^{(k)}, \mathbf{y}') \exp(\Delta^{(k)} / t_k)$
 If $U \leq p^{(k)}$, accept move: $\mathbf{y}^{(k+1)} \leftarrow \mathbf{y}'$
 If $U > p^{(k)}$, reject move: $\mathbf{y}^{(k+1)} \leftarrow \mathbf{y}^{(k)}$
end for
Output: $\hat{\mathbf{y}}_t(\theta) \approx \mathbf{y}^{(K)}$ (SA) or
 $\hat{\mathbf{y}}_t(\theta) = \mathbb{E}_{\pi_{\theta,t}}[Y] \approx \frac{1}{K} \sum_{k=1}^K \mathbf{y}^{(k)}$ (MH)

Algorithm 2 Neighborhood mixture MCMC

Inputs: $\theta \in \mathbb{R}^d, \mathbf{y}^{(0)} \in \mathcal{Y}, t, K \in \mathbb{N}, (\mathcal{N}_s, q_s)_{s=1}^S$
for $k = 0 : K$ **do**
 Sample a neighborhood system:
 $s \sim \mathcal{U}(Q(\mathbf{y}^{(k)}))$
 Sample a neighbor in $\mathcal{N}_s(\mathbf{y}^{(k)})$:
 $\mathbf{y}' \sim q_s(\mathbf{y}^{(k)}, \cdot)$
 $\alpha_s(\mathbf{y}^{(k)}, \mathbf{y}') \leftarrow \frac{|Q(\mathbf{y}^{(k)})| q_s(\mathbf{y}', \mathbf{y}^{(k)})}{|Q(\mathbf{y}')| q_s(\mathbf{y}^{(k)}, \mathbf{y}')} \text{ (MH)}$
 $U \sim \mathcal{U}([0, 1])$
 $\Delta^{(k)} \leftarrow \langle \theta, \mathbf{y}' \rangle + \varphi(\mathbf{y}') - \langle \theta, \mathbf{y}^{(k)} \rangle - \varphi(\mathbf{y}^{(k)})$
 $p^{(k)} \leftarrow \alpha_s(\mathbf{y}^{(k)}, \mathbf{y}') \exp(\Delta^{(k)} / t)$
 If $U \leq p^{(k)}$, accept move: $\mathbf{y}^{(k+1)} \leftarrow \mathbf{y}'$
 If $U > p^{(k)}$, reject move: $\mathbf{y}^{(k+1)} \leftarrow \mathbf{y}^{(k)}$
end for
Output: $\hat{\mathbf{y}}_t(\theta) = \mathbb{E}_{\pi_{\theta,t}}[Y] \approx \frac{1}{K} \sum_{k=1}^K \mathbf{y}^{(k)}$

Formally, we denote the neighborhood graph by $G_{\mathcal{N}} := (\mathcal{Y}, E_{\mathcal{N}})$, where edges are defined by $\mathcal{N}$. We assume the graph is undirected, i.e., $\mathbf{y}' \in \mathcal{N}(\mathbf{y})$ if and only if $\mathbf{y} \in \mathcal{N}(\mathbf{y}')$, and without self-loops – i.e., $\mathbf{y} \notin \mathcal{N}(\mathbf{y})$. A stochastic neighbor generating function is also provided, in the form of a proposal distribution $q(\mathbf{y}, \cdot)$ with support either equal to $\mathcal{N}(\mathbf{y})$ or $\mathcal{N}(\mathbf{y}) \cup \{\mathbf{y}\}$.

Link between simulated annealing and Metropolis-Hastings. A well-known example of local search heuristic is simulated annealing (SA) [26]. It is intimately related to Metropolis-Hastings (MH) [21], an instance of a MCMC algorithm. We provide a unified view of both in Algorithm 1.

The difference lies in the acceptance rule, which incorporates a proposal correction ratio for MH, and in the choice of the sequence of temperatures $(t_k)_{k \in \mathbb{N}}$. In the case of SA, it is chosen to verify $t_k \rightarrow 0$. In the case of MH, it is such that $t_k \equiv t$. In this case, the iterates $\mathbf{y}^{(k)}$ of Algorithm 1 follow a time-homogenous Markov chain on $\mathcal{Y}$, defined by the following transition kernel:

$$P_{\theta,t}(\mathbf{y}, \mathbf{y}') = \begin{cases} q(\mathbf{y}, \mathbf{y}') \min \left[1, \frac{q(\mathbf{y}', \mathbf{y})}{q(\mathbf{y}, \mathbf{y}')} \exp \left(\frac{\langle \theta, \mathbf{y}' \rangle + \varphi(\mathbf{y}') - \langle \theta, \mathbf{y} \rangle - \varphi(\mathbf{y})}{t} \right) \right] & \text{if } \mathbf{y}' \in \mathcal{N}(\mathbf{y}), \\ 1 - \sum_{\mathbf{y}'' \in \mathcal{N}(\mathbf{y})} P_{\theta,t_k}(\mathbf{y}, \mathbf{y}'') & \text{if } \mathbf{y}' = \mathbf{y}, \\ 0 & \text{else.} \end{cases} \quad (3)$$

In past work, the link between the two algorithms has primarily been used to show that SA converges to the exact MAP solution in the limit of infinite iterations [36, 17]. Under mild conditions – if the neighborhood graph $G_{\mathcal{N}}$ is connected and the chain is aperiodic – the iterates $\mathbf{y}^{(k)}$ of Algorithm 1 (MH case) converge in distribution to the Gibbs distribution (see Appendix C.1 for a proof):

$$\pi_{\theta,t}(\mathbf{y}) \propto \exp([\langle \theta, \mathbf{y} \rangle + \varphi(\mathbf{y})] / t). \quad (4)$$

Proposed layer. Algorithm 1 motivates us to define the combinatorial MCMC layer

$$\hat{\mathbf{y}}_t(\theta) := \mathbb{E}_{\pi_{\theta,t}}[Y], \quad (5)$$

where $\theta \in \mathbb{R}^d$ are logits and $t > 0$ is a temperature parameter, defaulting to $t = 1$. Naturally, the estimate of $\hat{\mathbf{y}}_t(\theta)$ returned by Algorithm 1 (MH case) is biased, as the Markov chain cannot perfectly mix in a finite number of iterations, except if it is initialized at $\pi_{\theta,t}$. In Section 4, we will show that this does not hinder the convergence of the proposed learning algorithms. The next proposition, proved in Appendix C.2, states some useful properties of the proposed layer.

Proposition 1. *Let $\theta \in \mathbb{R}^d$. We have the following properties:*

$$\hat{\mathbf{y}}_t(\theta) \in \text{relint}(\mathcal{C}), \quad \hat{\mathbf{y}}_t(\theta) \xrightarrow[t \rightarrow 0^+]{} \underset{\mathbf{y} \in \mathcal{Y}}{\operatorname{argmax}} \langle \theta, \mathbf{y} \rangle + \varphi(\mathbf{y}), \quad \text{and} \quad \hat{\mathbf{y}}_t(\theta) \xrightarrow[t \rightarrow \infty]{} \frac{1}{|\mathcal{Y}|} \sum_{\mathbf{y} \in \mathcal{Y}} \mathbf{y}.$$

Moreover, $\hat{\mathbf{y}}_t$ is differentiable and its Jacobian matrix is given by $J_{\theta} \hat{\mathbf{y}}_t(\theta) = \frac{1}{t} \operatorname{cov}_{\pi_{\theta,t}}[Y]$.

3.2 Mixing neighborhood systems

Central to local search algorithms in combinatorial optimization is the use of multiple neighborhood systems to more effectively explore the solution space [37, 10]. In this section, we propose an efficient way to incorporate such diversity of neighborhood systems into the combinatorial MCMC layer, while preserving the correct stationary distribution, by mixing the corresponding proposal distributions.

Let $(\mathcal{N}_s)_{s=1}^S$ be a set of different neighborhood systems. Typically, all neighborhood systems are not defined on all solutions $\mathbf{y} \in \mathcal{Y}$, so we note $Q(\mathbf{y}) \subseteq \llbracket 1, S \rrbracket$ the set of neighborhood systems defined on $\mathbf{y}$ (i.e., the set of allowed moves on $\mathbf{y}$). Let $(q_s)_{s \in Q(\mathbf{y})}$ be the corresponding proposal distributions, such that the support of $q_s(\mathbf{y}, \cdot)$ is either $\mathcal{N}_s(\mathbf{y})$ or $\mathcal{N}_s(\mathbf{y}) \cup \{\mathbf{y}\}$. Let $\bar{\mathcal{N}}$ be the aggregate neighborhood system defined by $\bar{\mathcal{N}} : \mathbf{y} \mapsto \bigcup_{s \in Q(\mathbf{y})} \mathcal{N}_s(\mathbf{y})$.

We assume that the individual Metropolis correction ratios $\tilde{\alpha}_s(\mathbf{y}, \mathbf{y}') := \frac{q_s(\mathbf{y}', \mathbf{y})}{q_s(\mathbf{y}, \mathbf{y}')}$ are tractable. The proposed procedure is summarized in Algorithm 2.

Proposition 2. *If each neighborhood graph $G_{\mathcal{N}_s}$ is undirected and without self-loops, and the aggregate neighborhood graph $G_{\bar{\mathcal{N}}}$ is connected, the iterations $\mathbf{y}^{(k)}$ produced by Algorithm 2 follow a Markov chain with unique stationary distribution $\pi_{\theta, t}$.*

See Appendix C.3 for the proof. Importantly, only the connectedness of the aggregate neighborhood graph $G_{\bar{\mathcal{N}}}$ is required. This allows us to combine neighborhood systems that could not connect $\mathcal{Y}$ if used individually, i.e., an irreducible Markov chain can be obtained by mixing the proposal distributions of reducible ones. Such an example is given with the moves defined in Table 2.

4 Loss functions and theoretical analysis

In this section, we derive and study loss functions for learning models using the proposed layer.

4.1 Negative log-likelihood and associated Fenchel-Young loss

We now show that the proposed layer $\hat{\mathbf{y}}_t(\theta)$ can be viewed as the solution of a regularized optimization problem on $\mathcal{C} = \text{conv}(\mathcal{Y})$. Let $A_t(\theta) := t \cdot \log \sum_{\mathbf{y} \in \mathcal{Y}} \exp([\langle \theta, \mathbf{y} \rangle + \varphi(\mathbf{y})] / t)$ be the cumulant function [53] associated to the exponential family defined by $\pi_{\theta, t}$, scaled by the temperature t . We define the regularization function Ω_t and the corresponding Fenchel-Young loss [8] as:

$$\Omega_t(\mu) := A_t^*(\mu) = \sup_{\theta \in \mathbb{R}^d} \langle \mu, \theta \rangle - A_t(\theta), \quad \text{and} \quad \ell_t(\theta; \mathbf{y}) := (\Omega_t)^*(\theta) + \Omega_t(\mathbf{y}) - \langle \theta, \mathbf{y} \rangle.$$

Since $\Omega_t = A_t^*$ is strictly convex on $\text{reint}(\mathcal{C})$ (see Appendix C.4 for a proof) and $\hat{\mathbf{y}}_t(\theta) = \nabla_{\theta} A_t(\theta)$, the proposed layer is the solution of the regularized optimization problem

$$\hat{\mathbf{y}}_t(\theta) = \underset{\mu \in \mathcal{C}}{\text{argmax}} \{ \langle \theta, \mu \rangle - \Omega_t(\mu) \},$$

the Fenchel-Young loss ℓ_t is differentiable, satisfies $\ell_t(\theta, \mathbf{y}) = 0 \Leftrightarrow \hat{\mathbf{y}}_t(\theta) = \mathbf{y}$, and has gradient $\nabla_{\theta} \ell_t(\theta; \mathbf{y}) = \hat{\mathbf{y}}_t(\theta) - \mathbf{y}$. It is therefore equivalent, up to a constant, to the negative log-likelihood loss, as we have $-\nabla_{\theta} \log \pi_{\theta, t}(\mathbf{y}) = (\hat{\mathbf{y}}_t(\theta) - \mathbf{y})/t$. Algorithms 1 and 2 can thus be used to perform maximum likelihood estimation, by returning a (biased) stochastic estimate of the gradient of ℓ_t .

4.2 Empirical risk minimization

In the supervised learning setting, we are given observations $(\mathbf{x}_i, \mathbf{y}_i)_{i=1}^N \in (\mathbb{R}^p \times \mathcal{Y})^N$, and want to fit a model $g_W : \mathbb{R}^p \rightarrow \mathbb{R}^d$ such that $\hat{\mathbf{y}}_t(g_W(\mathbf{x}_i)) \approx \mathbf{y}_i$. This is motivated by a generative model where, for some weights $W_0 \in \mathbb{R}^p$, the data is generated with $\mathbf{y}_i \sim \pi_{g_{W_0}(\mathbf{x}_i), t}$. We aim at minimizing the empirical risk L_N , defined below along with its exact gradient ∇L_N :

$$L_N(W) := \frac{1}{N} \sum_{i=1}^N \ell_t(g_W(\mathbf{x}_i); \mathbf{y}_i) \quad \text{and} \quad \nabla_W L_N(W) = \frac{1}{N} \sum_{i=1}^N J_W g_W(\mathbf{x}_i) (\hat{\mathbf{y}}_t(g_W(\mathbf{x}_i)) - \mathbf{y}_i).$$

Doubly stochastic gradient estimator. In practice, we cannot compute the exact gradient above. Using Algorithm 1 to get a MCMC estimate of $\hat{\mathbf{y}}_t(g_W(\mathbf{x}_i))$, we propose the following estimator:

$$\nabla_W L_N(W) \approx J_W g_W(\mathbf{x}_i) \left(\frac{1}{K} \sum_{k=1}^K \mathbf{y}_i^{(k)} - \mathbf{y}_i \right),$$

where $\mathbf{y}_i^{(k)}$ is the k -th iterate of Algorithm 1 with maximization direction $\boldsymbol{\theta}_i = g_W(\mathbf{x}_i)$ and temperature t . This estimator is doubly stochastic, since we sample both data points and iterations of Algorithm 1, and can be seamlessly used with batches. The term $J_W g_W(\mathbf{x}_i)$ is computed via autodiff.

Markov chain initialization. Following the contrastive divergence literature [22], in the supervised setting, we initialize the Markov chains at the corresponding ground-truth structure, by setting $\mathbf{y}_i^{(0)} = \mathbf{y}_i$. In the unsupervised setting, we use a persistent initialization method [50] instead.

4.3 Associated Fenchel-Young loss with a single MCMC iteration

In Section 4.1, we saw that there exists a Fenchel-Young loss (the negative log-likelihood loss) associated with the proposed layer $\hat{\mathbf{y}}_t$. However, it assumes that the number of MCMC iterations goes to infinity ($K \rightarrow \infty$). In practice, since we always use a finite number of iterations K , this means that our doubly stochastic gradient estimator is *biased* with respect to that loss.

We now show that when a single MCMC iteration is used ($K = 1$), there exists *another* target-dependent Fenchel-Young loss such that the stochastic gradient estimator is *unbiased* with respect to that loss. See Appendix C.7 for the construction of $\Omega_{\mathbf{y}}$ and the proof.

Proposition 3 (Existence of a Fenchel-Young loss when $K = 1$). *Let $\mathbf{p}_{\boldsymbol{\theta}, \mathbf{y}}^{(1)}$ denote the distribution of the first iterate of the Markov chain defined by the Markov transition kernel given in Eq. (3), with proposal distribution q and initialized at ground-truth $\mathbf{y} \in \mathcal{Y}$. There exists a target-dependent regularization function $\Omega_{\mathbf{y}}$ with the following properties: $\Omega_{\mathbf{y}}$ is $t/\mathbb{E}_{q(\mathbf{y}, \cdot)} \|\mathbf{Y} - \mathbf{y}\|_2^2$ -strongly convex; it is such that*

$$\mathbb{E}_{\mathbf{p}_{\boldsymbol{\theta}, \mathbf{y}}^{(1)}} [\mathbf{Y}] = \underset{\boldsymbol{\mu} \in \text{conv}(\mathcal{N}(\mathbf{y}) \cup \{\mathbf{y}\})}{\text{argmax}} \{ \langle \boldsymbol{\theta}, \boldsymbol{\mu} \rangle - \Omega_{\mathbf{y}}(\boldsymbol{\mu}) \}$$

and the Fenchel-Young loss $\ell_{\Omega_{\mathbf{y}}}$ generated by $\Omega_{\mathbf{y}}$ is $\mathbb{E}_{q(\mathbf{y}, \cdot)} \|\mathbf{Y} - \mathbf{y}\|_2^2/t$ -smooth in its first argument, and such that $\nabla_{\boldsymbol{\theta}} \ell_{\Omega_{\mathbf{y}}}(\boldsymbol{\theta}; \mathbf{y}) = \mathbb{E}_{\mathbf{p}_{\boldsymbol{\theta}, \mathbf{y}}^{(1)}} [\mathbf{Y}] - \mathbf{y}$.

A similar result in the unsupervised setting with data-based initialization is given in Proposition 6. Interestingly, these results contrast with prior work on the expected CD-1 update. Indeed, when applied with Gibbs sampling to train restricted Boltzmann machines, the latter was shown in Sutskever and Tieleman [49] not to be the gradient of any function – let alone a convex one.

4.4 Convergence analysis in the unsupervised setting

In the unsupervised setting, we are only given observations $(\mathbf{y}_i)_{i=1}^N \in \mathcal{Y}^N$ and want to fit a model $\pi_{\boldsymbol{\theta}, t}$, motivated by an underlying generative model such that $\mathbf{y}_i \sim \pi_{\boldsymbol{\theta}_0, t}$ for an unknown true parameter $\boldsymbol{\theta}_0$. We assume here that $\mathcal{C} = \text{conv}(\mathcal{Y})$ is of full dimension in $\mathbb{R}^d$. If not, such a model is specified only up to vectors orthogonal to the direction of the smallest affine subspace spanned by $\mathcal{C}$. Indeed, if $\boldsymbol{\mu} \in \mathbb{R}^d$ is such that $\forall \mathbf{y} \in \mathcal{Y}, \langle \boldsymbol{\mu}, \mathbf{y} \rangle = C \in \mathbb{R}$, then we have $\pi_{\boldsymbol{\theta}, t} = \pi_{\boldsymbol{\theta} + \boldsymbol{\mu}, t}$.

We have the corresponding empirical L_N and population $L_{\boldsymbol{\theta}_0}$ Fenchel-Young losses:

$$L_N(\boldsymbol{\theta}; \mathbf{y}_1, \dots, \mathbf{y}_N) := \frac{1}{N} \sum_{i=1}^N \ell_t(\boldsymbol{\theta}; \mathbf{y}_i), \quad L_{\boldsymbol{\theta}_0}(\boldsymbol{\theta}) := \mathbb{E}_{(\mathbf{y}_i)_{i=1}^N \sim (\pi_{\boldsymbol{\theta}_0, t})^{\otimes N}} [L_N(\boldsymbol{\theta}; \mathbf{y}_1, \dots, \mathbf{y}_N)],$$

which are minimized for $\boldsymbol{\theta}$ such that $\hat{\mathbf{y}}_t(\boldsymbol{\theta}) = \bar{\mathbf{Y}}_N := \frac{1}{N} \sum_{i=1}^N \mathbf{y}_i$, and for $\boldsymbol{\theta}$ such that $\hat{\mathbf{y}}_t(\boldsymbol{\theta}) = \hat{\mathbf{y}}_t(\boldsymbol{\theta}_0)$, respectively. Let $\boldsymbol{\theta}_N^*$ as the minimizer of the empirical loss L_N . For it to be defined, we assume N is large enough to have $\bar{\mathbf{Y}}_N \in \text{relint}(\mathcal{C})$ (which is always possible as $\pi_{\boldsymbol{\theta}_0, t}$ has dense support on $\mathcal{Y}$). A slight variation on Proposition 4.1 in Berthet et al. [6], proved in Appendix C.5, gives the following asymptotic normality as $N \rightarrow \infty$.

Proposition 4 (Convergence of the empirical loss minimizer to the true parameter).

$$\sqrt{N}(\boldsymbol{\theta}_N^* - \boldsymbol{\theta}_0) \xrightarrow[N \rightarrow \infty]{\mathcal{D}} \mathcal{N}\left(\mathbf{0}, t^2 \text{cov}_{\pi_{\boldsymbol{\theta}_0, t}}[Y]^{-1}\right).$$

We now consider the sample size as fixed to N samples, and define $\hat{\boldsymbol{\theta}}_n$ as the n -th iterate of the following stochastic gradient algorithm:

$$\hat{\boldsymbol{\theta}}_{n+1} = \hat{\boldsymbol{\theta}}_n + \gamma_{n+1} \left[\bar{Y}_N - \frac{1}{K_{n+1}} \sum_{k=1}^{K_{n+1}} \mathbf{y}^{(n+1, k)} \right], \quad (6)$$

where $\mathbf{y}^{(n+1, k)}$ is the k -th iterate of Algorithm 1 with temperature t , maximization direction $\hat{\boldsymbol{\theta}}_n$, and initialized at $\mathbf{y}^{(n+1, 1)} = \mathbf{y}^{(n, K_n)}$. This initialization corresponds to the persistent contrastive divergences (PCD) algorithm [50], and is further discussed in Appendix B.2.

Proposition 5 (Convergence of the stochastic gradient estimate). *Suppose the following assumptions on the step sizes $(\gamma_n)_{n \geq 1}$, sample sizes $(K_n)_{n \geq 1}$, and proposal distribution q hold:*

- $\gamma_n = an^{-b}$, with $b \in [\frac{1}{2}, 1]$ and $a > 0$,
- $K_{n+1} > \left\lceil 1 + a' \exp\left(\frac{8R_C}{t} \cdot \|\hat{\boldsymbol{\theta}}_n\|\right) \right\rceil$ with $a' > 0$ and $R_C = \max_{\mathbf{y} \in \mathcal{Y}} \|\mathbf{y}\|$,
- $\frac{1}{\sqrt{K_n}} - \frac{1}{\sqrt{K_{n-1}}} \leq a''n^{-c}$, with $a'' > 0$ and $c > 1 - \frac{b}{2}$,
- $q(\mathbf{y}, \mathbf{y}') = \begin{cases} \frac{1}{2d^*} & \text{if } \mathbf{y}' \in \mathcal{N}(\mathbf{y}), \\ 1 - \frac{d(\mathbf{y})}{2d^*} & \text{if } \mathbf{y}' = \mathbf{y}, \\ 0 & \text{else,} \end{cases}$
where $d(\mathbf{y}) := |\mathcal{N}(\mathbf{y})|$ is the degree of $\mathbf{y}$ in G_N , and $d^* := \max_{\mathbf{y} \in \mathcal{Y}} d(\mathbf{y})$.

Then, we have the almost sure convergence $\hat{\boldsymbol{\theta}}_n \xrightarrow{a.s.} \boldsymbol{\theta}_N^*$ of the iterates $\hat{\boldsymbol{\theta}}_n$ defined by Eq. (6).

See Appendix C.6 for the proof. The assumptions on q are used for obtaining a closed-form convergence rate bound for the Markov chain, using graph-based geometric bounds [23].

5 Experiments on dynamic vehicle routing

Overview of the challenge. We evaluate the proposed approach on a large-scale, ML-enriched combinatorial optimization problem: the *EURO Meets NeurIPS 2022 Vehicle Routing Competition* [27]. In this dynamic vehicle routing problem with time windows (DVRPTW), requests arrive continuously throughout a planning horizon, which is partitioned into a series of delivery waves $\mathcal{W} = \{[\tau_0, \tau_1], [\tau_1, \tau_2], \dots, [\tau_{|\mathcal{W}|-1}, \tau_{|\mathcal{W}|}]\}$.

At the start of each wave ω , a dispatching and vehicle routing problem must be solved for the set of requests $\mathcal{R}^\omega$ specific to that wave (in which we include the depot D), encoded into the system state $\mathbf{x}^\omega$. We note $\mathcal{Y}(\mathbf{x}^\omega)$ the set of feasible decisions associated to state $\mathbf{x}^\omega$.

A feasible solution $\mathbf{y}^\omega \in \mathcal{Y}(\mathbf{x}^\omega)$ must contain all requests that must be dispatched before τ_ω (the rest are postponable), allow each of its routes to visit the requests they dispatch within their respective time windows, and be such that the cumulative customer demand on each of its routes does not exceed a given vehicle capacity. It is encoded thanks to a vector $(y_{i,j}^\omega)_{i,j \in \mathcal{R}^\omega}$, where $y_{i,j}^\omega = 1$ if the solution contains the directed route segment from i to j , and $y_{i,j}^\omega = 0$ otherwise. The set of requests $\mathcal{R}^{\omega+1}$ is obtained by removing all requests dispatched by the chosen solution $\mathbf{y}^\omega$ from $\mathcal{R}^\omega$ and adding all new requests which arrived between τ_ω and $\tau_{\omega+1}$.

The aim of the challenge is to find an optimal policy $f: \mathcal{X} \rightarrow \mathcal{Y}$ assigning decisions $\mathbf{y}^\omega \in \mathcal{Y}(\mathbf{x}^\omega)$ to system states $\mathbf{x}^\omega \in \mathcal{X}$. This can be cast as a reinforcement learning problem:

$$\min_{\Psi} \mathbb{E}[c_{\mathcal{W}}(f)], \quad \text{with} \quad c_{\mathcal{W}}(f) := \sum_{\omega \in \mathcal{W}} c(f(\mathbf{x}^\omega)),$$

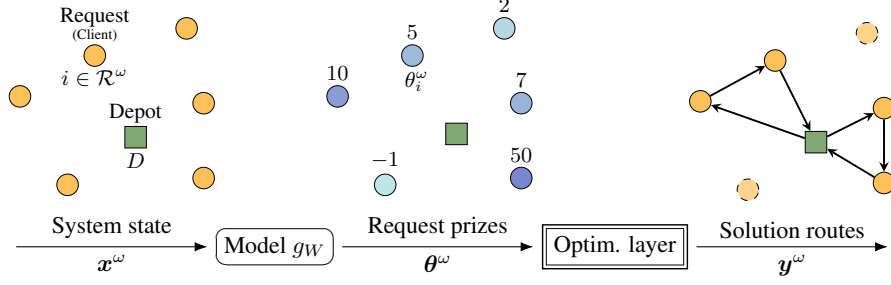


Figure 1: Overview of the vehicle routing pipeline, represented at request wave ω .

where $c : \mathbf{y}^\omega \mapsto \sum_{i,j \in \mathcal{R}^\omega} c_{i,j} y_{i,j}^\omega$ gives the routing cost of $\mathbf{y}^\omega \in \mathcal{Y}^\omega$ and where $c_{i,j} \geq 0$ is the routing cost from i to j . The expectation is taken over full problem instances.

Reduction to supervised learning. We follow the method of [4], which was the winning approach for the challenge. Central to this approach is the concept of prize-collecting dynamic vehicle routing problem with time windows (PC-VRPTW). In this setting, each request $i \in \mathcal{R}^\omega$ is assigned an artificial *prize* $\theta_i^\omega \in \mathbb{R}$, that reflects the benefit of serving it. The prize of the depot D is set to $\theta_D^\omega = 0$. The objective is then to identify a set of routes that maximizes the total prize collected while minimizing the associated travel costs. The model g_W predicts the prize vector $\theta^\omega = g_W(\mathbf{x}^\omega)$. Denoting $\varphi(\mathbf{y}) := -\langle c, \mathbf{y} \rangle$, the corresponding optimization problem can be written as

$$\hat{\mathbf{y}}(\theta^\omega) = \operatorname{argmax}_{\mathbf{y} \in \mathcal{Y}(\mathbf{x}^\omega)} \sum_{i,j \in \mathcal{R}^\omega} \theta_j^\omega y_{i,j} - \sum_{i,j \in \mathcal{R}^\omega} c_{i,j} y_{i,j} = \langle \theta^\omega, \mathbf{y} \rangle + \varphi(\mathbf{y}). \quad (7)$$

The overall pipeline is summarized in Fig. 1. Following [4], we approximately solve the problem in Eq. (7) using the prize-collecting HGS heuristic (PC-HGS), a variant of hybrid genetic search (HGS) [51]. We denote this approximate solver $\tilde{\mathbf{y}} \approx \hat{\mathbf{y}}$, so that their proposed policy decomposes as $f_W := \tilde{\mathbf{y}} \circ g_W$. The ground-truth routes are created by using an anticipative strategy, i.e., by solving multiple instances where all future information is revealed from the start, and the requests' arrival times information is translated into time windows (thus removing the dynamic aspect of the problem). This anticipative policy, which we note f^* (which cannot be attained as it needs unavailable information) is thus the target policy imitated by the model – see Appendix B.4 for more details.

Perturbation-based baseline. In [4], a perturbation-based method [6] was used. This method is based on injecting noise in the PC-HGS solver $\tilde{\mathbf{y}}$. Similarly to our approach, the parameters W can then be learned using a Fenchel-Young loss, since the loss is minimized when the perturbed solver correctly predicts the ground truth. However, since $\tilde{\mathbf{y}}$ is not an exact solver, all theoretical learning guarantees associated with this method (e.g., correctness of the gradients) no longer hold.

Proposed approach. Our proposed approach instead uses the Fenchel-Young loss associated with the proposed layer, which is minimized when the proposed layer correctly predicts the ground-truth. At inference time, however, we use $f_W := \tilde{\mathbf{y}} \circ g_W$. We use a mixture of proposals, as defined in Algorithm 2. To design each proposal q_s , we build randomized versions of moves specifically designed for the prize-collecting dynamic vehicle routing problem with time windows. More precisely, we base our proposals on moves used in the local search part of the PC-HGS algorithm, which are summarized in Table 2. The details of turning these moves into proposal distributions with tractable individual correction ratios are given in Appendix B.3.

We evaluate three different initialization methods: (i) initialize $\mathbf{y}^{(0)}$ by constructing routes dispatching random requests, (ii) initialize $\mathbf{y}^{(0)}$ to the ground-truth solution, (iii) initialize $\mathbf{y}^{(0)}$ by starting from the dataset ground-truth and applying a heuristic initialization algorithm to improve it. This heuristic initialization, similar to a short local search, is also used by the PC-HGS algorithm $\tilde{\mathbf{y}}$, and is set to take up to half the time allocated to the layer (a limit it does not reach in practice).

Performance metric. As the Fenchel-Young loss ℓ_t actually minimized is intractable to compute exactly, we only use the challenge metric. More precisely, we measure the cost relative to that of the anticipative baseline, $\frac{c_W(f_W) - c_W(f^*)}{c_W(f^*)}$, which we average over a test dataset of unseen instances.

Table 2: Local search moves used for creating neighborhoods in our vehicle routing experiments.

Name	Description
<code>relocate</code>	Removes a single request from its route and re-inserts it at a different position in the solution.
<code>relocate pair</code>	Removes a pair of consecutive requests from their route and re-inserts them at a different position in the solution.
<code>swap</code>	Exchanges the position of two requests in the solution.
<code>swap pair</code>	Exchanges the positions of two pairs of consecutive requests in the solution.
<code>2-opt</code>	Reverses a route segment.
<code>serve request</code>	Inserts a currently unserved request into the solution.
<code>remove request</code>	Removes a request from the solution.

Results. In Fig. 2, we observe that the initialization method plays an important role, and the ground-truth-based ones greatly outperform the random one. We observe that the number of Markov iterations K is an important performance factor. Interestingly, the ground-truth initialization significantly improves the learning process for small K .

In Table 3, we compare training methods with fixed compute time budget for the layer (perturbed solver or proposed MCMC approach), which is by far the main computational bottleneck. The models are selected using a validation set and evaluated on the test set. We observe that the proposed approach significantly outperforms the perturbation-based method [6] using $\tilde{y}$ in low time limit regimes, thus allowing for faster and more efficient training. Full experimental details and additional results on the impact of temperature are given in Appendix B.4.

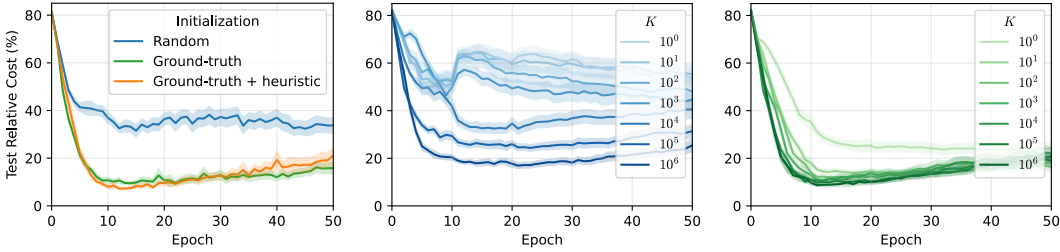


Figure 2: Test relative cost (%). **Left:** varying initialization method. **Center:** varying number of Markov iterations K with random initialization. **Right:** varying K with ground-truth initialization.

Table 3: Best test relative cost (%) for different training methods and time limits.

Time limit (ms)	1	5	10	50	100	1000
Perturbed inexact oracle	65.2 ± 5.8	13.1 ± 3.4	8.7 ± 1.9	6.5 ± 1.1	6.3 ± 0.76	5.5 ± 0.4
Proposed ($y^{(0)}=y$)	10.0 ± 1.7	12.0 ± 2.6	11.8 ± 2.8	9.1 ± 1.7	8.4 ± 1.7	7.7 ± 1.1
Proposed ($y^{(0)}=y+\text{heuristic}$)	7.8 ± 0.8	7.2 ± 0.6	6.3 ± 0.7	6.2 ± 0.8	5.9 ± 0.7	5.9 ± 0.6

6 Conclusion, limitations and future work

In this paper, we introduced a principled framework for integrating NP-hard combinatorial optimization layers into neural networks without relying on exact solvers. Our approach adapts neighborhood systems from the metaheuristics community, to design structure-aware proposal distributions for combinatorial MCMC. This leads to significant training speed-ups, which are crucial for tackling larger problem instances, especially in operations research where scaling up leads to substantial value creation by reducing marginal costs. However, our framework requires opening the solver’s blackbox, and correction ratios must be implemented in the acceptance rule of the local search to guarantee convergence to the correct stationary distribution. In future work, we plan to extend our framework to large neighborhood search algorithms, which are local search heuristics that leverage neighborhood-wise exact optimization oracles.

References

- [1] Akshay Agrawal, Brandon Amos, Shane Barratt, Stephen Boyd, Steven Diamond, and Zico Kolter. Differentiable convex optimization layers, 2019. URL <https://arxiv.org/abs/1910.12430>.
- [2] Kareem Ahmed, Zhe Zeng, Mathias Niepert, and Guy Van den Broeck. SIMPLE: A gradient estimator for k -subset sampling, 2024. URL <http://arxiv.org/abs/2210.01941>.
- [3] Brandon Amos and J Zico Kolter. Optnet: Differentiable optimization as a layer in neural networks. In *International Conference on Machine Learning*, pages 136–145. PMLR, 2017.
- [4] Léo Baty, Kai Jungel, Patrick S. Klein, Axel Parmentier, and Maximilian Schiffer. Combinatorial optimization enriched machine learning to solve the dynamic vehicle routing problem with time windows, 2023. URL <http://arxiv.org/abs/2304.00789>.
- [5] Yoshua Bengio, Andrea Lodi, and Antoine Prouvost. Machine learning for combinatorial optimization: a methodological tour d’horizon, 2020. URL <http://arxiv.org/abs/1811.06128>.
- [6] Quentin Berthet, Mathieu Blondel, Olivier Teboul, Marco Cuturi, Jean-Philippe Vert, and Francis Bach. Learning with differentiable perturbed optimizers, 2020. URL <http://arxiv.org/abs/2002.08676>.
- [7] Mathieu Blondel and Vincent Roulet. The Elements of Differentiable Programming. *arXiv preprint arXiv:2403.14606*, 2024.
- [8] Mathieu Blondel, André F. T. Martins, and Vlad Niculae. Learning with fenchel-young losses, 2020. URL <http://arxiv.org/abs/1901.02324>.
- [9] Mathieu Blondel, Quentin Berthet, Marco Cuturi, Roy Frostig, Stephan Hoyer, Felipe Llinares-López, Fabian Pedregosa, and Jean-Philippe Vert. Efficient and modular implicit differentiation. *Advances in neural information processing systems*, 35:5230–5242, 2022.
- [10] Christian Blum and Andrea Roli. Metaheuristics in combinatorial optimization: Overview and conceptual comparison. 35(3):268–308, 2003. ISSN 0360-0300. doi: 10.1145/937503.937505. URL <https://doi.org/10.1145/937503.937505>.
- [11] Miguel A Carreira-Perpiñán and Geoffrey Hinton. On contrastive divergence learning. In *International Workshop on Artificial Intelligence and Statistics*, pages 33–40. PMLR, 2005. URL <https://proceedings.mlr.press/r5/carreira-perpinan05a.html>.
- [12] Bor-Liang Chen and Ko-Wei Lih. Hamiltonian uniform subset graphs. 42(3):257–263, 1987. ISSN 0095-8956. doi: 10.1016/0095-8956(87)90044-X. URL <https://www.sciencedirect.com/science/article/pii/009589568790044X>.
- [13] John M. Danskin. The theory of max-min, with applications. 14(4):641–664, 1966. ISSN 0036-1399. doi: 10.1137/0114053. URL <https://epubs.siam.org/doi/abs/10.1137/0114053>.
- [14] Priya Donti, Brandon Amos, and J Zico Kolter. Task-based end-to-end model learning in stochastic optimization. In *Advances in Neural Information Processing Systems*, volume 30. Curran Associates, Inc., 2017.
- [15] Yilun Du and Igor Mordatch. Implicit generation and generalization in energy-based models, 2020. URL <https://arxiv.org/abs/1903.08689>.
- [16] Yilun Du, Shuang Li, Joshua Tenenbaum, and Igor Mordatch. Improved contrastive divergence training of energy based models, 2021. URL <https://arxiv.org/abs/2012.01316>.
- [17] Ulrich Faigle and Rainer Schrader. On the convergence of stationary distributions in simulated annealing algorithms. 27(4):189–194, 1988. ISSN 0020-0190. doi: 10.1016/0020-0190(88)90024-5. URL <https://www.sciencedirect.com/science/article/pii/0020019088900245>.

- [18] Ari Freedman. CONVERGENCE THEOREM FOR FINITE MARKOV CHAINS. 2017. URL <https://www.semanticscholar.org/paper/CONVERGENCE-THEOREM-FOR-FINITE-MARKOV-CHAINS-%E2%8B%82t/65f7c092bd9c59cbbc88dd69266d39cd79840648>.
- [19] Michel Gendreau, Jean-Yves Potvin, et al. *Handbook of metaheuristics*, volume 2. Springer, 2010.
- [20] Will Grathwohl, Kevin Swersky, Milad Hashemi, David Duvenaud, and Chris J. Maddison. Oops i took a gradient: Scalable sampling for discrete distributions, 2021. URL <https://arxiv.org/abs/2102.04509>.
- [21] W. K. Hastings. Monte carlo sampling methods using markov chains and their applications. *Biometrika*, 57(1):97–109, 1970. ISSN 00063444, 14643510. URL <http://www.jstor.org/stable/2334940>.
- [22] Geoffrey E. Hinton. Training products of experts by minimizing contrastive divergence. 2000. URL <https://www.semanticscholar.org/paper/Training-Products-of-Experts-by-Minimizing-Hinton/9360e5ce9c98166bb179ad479a9d2919ff13d022>.
- [23] Salvatore Ingrassia. On the rate of convergence of the metropolis algorithm and gibbs sampler by geometric bounds. 4(2):347–389, 1994. ISSN 1050-5164. URL <https://www.jstor.org/stable/2245161>.
- [24] Gareth A. Jones. Automorphisms and regular embeddings of merged johnson graphs. 26(3):417–435, 2005. ISSN 0195-6698. doi: 10.1016/j.ejc.2004.01.012. URL <https://www.sciencedirect.com/science/article/pii/S0195669804000630>.
- [25] Diederik P. Kingma and Jimmy Ba. Adam: A method for stochastic optimization, 2017. URL <http://arxiv.org/abs/1412.6980>.
- [26] S. Kirkpatrick, C. D. Gelatt, and M. P. Vecchi. Optimization by simulated annealing. *Science*, 220(4598):671–680, 1983. doi: 10.1126/science.220.4598.671. URL <https://www.science.org/doi/abs/10.1126/science.220.4598.671>.
- [27] Wouter Kool, Laurens Bliet, Danilo Numeroso, Yingqian Zhang, Tom Catshoek, Kevin Tierney, Thibaut Vidal, and Joaquim Gromicho. The EURO meets NeurIPS 2022 vehicle routing competition. In *Proceedings of the NeurIPS 2022 Competitions Track*, pages 35–49. PMLR, 2023. URL <https://proceedings.mlr.press/v220/kool23a.html>.
- [28] Rahul G. Krishnan, Simon Lacoste-Julien, and David Sontag. Barrier frank-wolfe for marginal inference, 2015. URL <https://arxiv.org/abs/1511.02124>.
- [29] John D. Lafferty, Andrew McCallum, and Fernando C. N. Pereira. Conditional random fields: Probabilistic models for segmenting and labeling sequence data. In *Proceedings of the Eighteenth International Conference on Machine Learning*, ICML '01, page 282–289, San Francisco, CA, USA, 2001. Morgan Kaufmann Publishers Inc. ISBN 1558607781.
- [30] Yann Lecun, Sumit Chopra, Raia Hadsell, Marc Aurelio Ranzato, and Fu Jie Huang. *A tutorial on energy-based learning*. MIT Press, 2006.
- [31] Zhifei Li and Jason Eisner. First- and second-order expectation semirings with applications to minimum-risk training on translation forests. In Philipp Koehn and Rada Mihalcea, editors, *Proceedings of the 2009 Conference on Empirical Methods in Natural Language Processing*, pages 40–51. Association for Computational Linguistics, August 2009. URL <https://aclanthology.org/D09-1005/>.
- [32] Neal Madras and Dana Randall. Markov chain decomposition for convergence rate analysis. 12(2):581–606, 2002. ISSN 1050-5164, 2168-8737. doi: 10.1214/aoap/1026915617. URL <https://projecteuclid.org/journals/annals-of-applied-probability/volume-12/issue-2/Markov-chain-decomposition-for-convergence-rate-analysis/10.1214/aoap/1026915617.full>.

- [33] Jayanta Mandi and Tias Guns. Interior point solving for LP-based prediction+optimisation, 2020. URL <http://arxiv.org/abs/2010.13943>.
- [34] Jayanta Mandi, James Kotary, Senne Berden, Maxime Mulamba, Victor Bucarey, Tias Guns, and Ferdinando Fioretto. Decision-focused learning: Foundations, state of the art, benchmark and future opportunities. 80:1623–1701, 2024. ISSN 1076-9757. doi: 10.1613/jair.1.15320. URL <http://arxiv.org/abs/2307.13565>.
- [35] Arthur Mensch and Mathieu Blondel. Differentiable dynamic programming for structured prediction and attention, 2018. URL <https://arxiv.org/abs/1802.03676>.
- [36] Debasis Mitra, Fabio Romeo, and Alberto Sangiovanni-Vincentelli. Convergence and finite-time behavior of simulated annealing. *Advances in Applied Probability*, 18(3):747–771, 1986. ISSN 0001-8678. doi: 10.2307/1427186. URL <https://www.jstor.org/stable/1427186>.
- [37] Nenad Mladenović and Pierre Hansen. Variable neighborhood search. *Computers & operations research*, 24(11):1097–1100, 1997.
- [38] Volodymyr Mnih, Hugo Larochelle, and Geoffrey E. Hinton. Conditional restricted boltzmann machines for structured output prediction, 2012. URL <http://arxiv.org/abs/1202.3748>.
- [39] Vlad Niculae, André F. T. Martins, Mathieu Blondel, and Claire Cardie. Sparsemap: Differentiable sparse structured inference, 2018. URL <https://arxiv.org/abs/1802.04223>.
- [40] Axel Parmentier. Learning structured approximations of combinatorial optimization problems. *arXiv preprint arXiv:2107.04323*, 2021.
- [41] Axel Parmentier. Learning to approximate industrial problems by operations research classic problems. *Operations Research*, 70(1):606–623, 2022.
- [42] Benjamin Rhodes and Michael Gutmann. Enhanced gradient-based MCMC in discrete spaces, 2022. URL <http://arxiv.org/abs/2208.00040>.
- [43] Fred J. Rispoli. The graph of the hypersimplex, 2008. URL <http://arxiv.org/abs/0811.2981>.
- [44] R. Tyrrell Rockafellar. *Convex Analysis*. Princeton University Press, 1970. ISBN 9780691015866. URL <http://www.jstor.org/stable/j.ctt14bs1ff>.
- [45] Utsav Sadana, Abhilash Chenreddy, Erick Delage, Alexandre Forel, Emma Frejinger, and Thibaut Vidal. A survey of contextual optimization methods for decision making under uncertainty, 2024. URL <http://arxiv.org/abs/2306.10374>.
- [46] Yang Song and Diederik P. Kingma. How to train your energy-based models, 2021. URL <https://arxiv.org/abs/2101.03288>.
- [47] Haoran Sun, Hanjun Dai, Bo Dai, Haomin Zhou, and Dale Schuurmans. Discrete langevin sampler via wasserstein gradient flow, 2023. URL <http://arxiv.org/abs/2206.14897>.
- [48] Haoran Sun, Katayoon Goshvadi, Azade Nova, Dale Schuurmans, and Hanjun Dai. Revisiting sampling for combinatorial optimization. In *Proceedings of the 40th International Conference on Machine Learning*, pages 32859–32874. PMLR, 2023. URL <https://proceedings.mlr.press/v202/sun23c.html>.
- [49] Ilya Sutskever and Tijmen Tieleman. On the convergence properties of contrastive divergence. In *Proceedings of the Thirteenth International Conference on Artificial Intelligence and Statistics*, pages 789–795. JMLR Workshop and Conference Proceedings, 2010. URL <https://proceedings.mlr.press/v9/sutskever10a.html>.
- [50] Tijmen Tieleman. Training restricted boltzmann machines using approximations to the likelihood gradient. In *Proceedings of the 25th international conference on Machine learning*, ICML ’08, pages 1064–1071. Association for Computing Machinery, 2008. ISBN 9781605582054. doi: 10.1145/1390156.1390290. URL <https://doi.org/10.1145/1390156.1390290>.

- [51] Thibaut Vidal. Hybrid genetic search for the cvrp: Open-source implementation and swap* neighborhood. *Computers & Operations Research*, 140:105643, April 2022. ISSN 0305-0548. doi: 10.1016/j.cor.2021.105643. URL <http://dx.doi.org/10.1016/j.cor.2021.105643>.
- [52] Marin Vlastelica, Anselm Paulus, Vít Musil, Georg Martius, and Michal Rolínek. Differentiation of blackbox combinatorial solvers, 2020. URL <http://arxiv.org/abs/1912.02175>.
- [53] Martin J. Wainwright and Michael I. Jordan. Graphical models, exponential families, and variational inference. 1(1):1–305, 2008. ISSN 1935-8237, 1935-8245. doi: 10.1561/22000000001. URL <https://www.nowpublishers.com/article/Details/MAL-001>.
- [54] Laurent Younes. Stochastic gradient estimation strategies for markov random fields. In *Bayesian Inference for Inverse Problems*, volume 3459, pages 315–325. SPIE, 1998. doi: 10.1117/12.323811. URL <https://www.spiedigitallibrary.org/conference-proceedings-of-spie/3459/0000/Stochastic-gradient-estimation-strategies-for-Markov-random-fields/10.1117/12.323811.full>.
- [55] Ruqi Zhang, Xingchao Liu, and Qiang Liu. A langevin-like sampler for discrete distributions, 2022. URL <https://arxiv.org/abs/2206.09914>.

A Experiments on empirical convergence of gradients and parameters

In this section, we evaluate the proposed approach on two discrete output spaces: sets and κ -subsets. These output spaces are for instance useful for multilabel classification. We focus on these output spaces because the exact MAP and marginal inference oracles are available, allowing us to compare our gradient estimators to exact gradients.

A.1 Polytopes and corresponding oracles

The vertex set of the first polytope is the set of binary vectors in $\mathbb{R}^d$, which we denote $\mathcal{Y}^d := \{0, 1\}^d$, and $\text{conv}(\mathcal{Y}^d) = [0, 1]^d$ is the “hypercube”. The vertex set of the second is the set of binary vectors with exactly κ ones and $d - \kappa$ zeros (with $0 < \kappa < d$),

$$\mathcal{Y}_\kappa^d := \{\mathbf{y} \in \{0, 1\}^d : \langle \mathbf{y}, \mathbf{1} \rangle = \kappa\},$$

and $\text{conv}(\mathcal{Y}_\kappa^d)$ is referred to as “top- κ polytope” or “hypersimplex”. Although these polytopes would not provide relevant use cases of the proposed approach in practice, since exact marginal inference oracles are available (see below), they allow us to compare the Fenchel-Young loss value and gradient estimated by our algorithm to their true value.

Marginal inference. For the hypercube, we have:

$$\begin{aligned} \mathbb{E}_{\pi_{\theta, t}}[Y_i] &= \sum_{\mathbf{y} \in \mathcal{Y}^d} \frac{\exp(\langle \boldsymbol{\theta}, \mathbf{y} \rangle / t)}{\sum_{\mathbf{y}' \in \mathcal{Y}^d} \exp(\langle \boldsymbol{\theta}, \mathbf{y}' \rangle / t)} y_i \\ &= \sum_{\mathbf{y} \in \{0, 1\}^d} \frac{\exp\left(\sum_{j=1}^d \theta_j y_j / t\right)}{\sum_{\mathbf{y}' \in \{0, 1\}^d} \exp\left(\sum_{j=1}^d \theta_j y'_j / t\right)} y_i \\ &= \sum_{y_i \in \{0, 1\}} \sum_{\mathbf{y}_{-i} \in \{0, 1\}^{d-1}} \frac{\exp\left(\theta_i y_i / t + \sum_{j \neq i} \theta_j y_j / t\right)}{\sum_{y'_i \in \{0, 1\}} \sum_{\mathbf{y}'_{-i} \in \{0, 1\}^{d-1}} \exp\left(\theta_i y'_i / t + \sum_{j \neq i} \theta_j y'_j / t\right)} y_i \\ &= \sum_{y_i \in \{0, 1\}} \frac{\exp(\theta_i y_i / t)}{\sum_{y'_i \in \{0, 1\}} \exp(\theta_i y'_i / t)} y_i \sum_{\mathbf{y}_{-i} \in \{0, 1\}^{d-1}} \frac{\exp\left(\sum_{j \neq i} \theta_j y_j / t\right)}{\sum_{\mathbf{y}'_{-i} \in \{0, 1\}^{d-1}} \exp\left(\sum_{j \neq i} \theta_j y'_j / t\right)} \\ &= \sum_{y_i \in \{0, 1\}} \frac{\exp(\theta_i y_i / t)}{\sum_{y'_i \in \{0, 1\}} \exp(\theta_i y'_i / t)} y_i \\ &= \frac{0 \cdot \exp(0) + 1 \cdot \exp(\theta_i / t)}{\exp(0) + \exp(\theta_i / t)} \\ &= \frac{\exp(\theta_i / t)}{1 + \exp(\theta_i / t)} \\ &= \sigma\left(\frac{\theta_i}{t}\right), \end{aligned}$$

which gives $\mathbb{E}_{\pi_{\theta,t}}[Y] = \sigma\left(\frac{\theta}{t}\right)$, where the logistic sigmoid function σ is applied component-wise. The cumulant function is also tractable, as we have

$$\begin{aligned}
\log \sum_{\mathbf{y} \in \mathcal{Y}^d} \exp(\langle \boldsymbol{\theta}, \mathbf{y} \rangle / t) &= \log \sum_{\mathbf{y} \in \{0,1\}^d} \exp\left(\sum_{i=1}^d \theta_i y_i / t\right) \\
&= \log \sum_{\mathbf{y}_1=0}^1 \sum_{\mathbf{y}_2=0}^1 \cdots \sum_{\mathbf{y}_d=0}^1 \exp\left(\sum_{i=1}^d \theta_i y_i / t\right) \\
&= \log \prod_{i=1}^d \sum_{y_i=0}^1 \exp(\theta_i y_i / t) \\
&= \log \prod_{i=1}^d (\exp(0) + \exp(\theta_i / t)) \\
&= \log \prod_{i=1}^d (1 + \exp(\theta_i / t)) \\
&= \sum_{i=1}^d \log(1 + \exp(\theta_i / t)).
\end{aligned}$$

Another way to derive this is via the Fenchel conjugate.

For the top- κ polytope, such closed-form formulas do not exist for the cumulant and its gradient. However, we implement them with dynamic programming, by viewing the top- κ MAP problem as a 0/1-knapsack problem with constant item weights, and by changing the $(\max, +)$ semiring into a $(\text{LSE}, +)$ semiring. This returns the cumulant function, and we leverage PyTorch's automatic differentiation framework to compute its gradient. This simple implementation allows us to compute true Fenchel-Young losses values and their gradients in $\mathcal{O}(d\kappa)$ time and space complexity.

Sampling. For the hypercube, sampling from the Gibbs distribution on $\mathcal{Y}^d$ has closed form. Indeed, the latter is fully factorized, and we can sample $\mathbf{y} \sim \pi_{\theta,t}$ by sampling independently each component with $y_i \sim \text{Bern}(\sigma(\theta_i/t))$. Sampling from $\pi_{\theta,t}$ is also possible on $\mathcal{Y}_{\kappa}^d$, by sampling coordinates iteratively using the dynamic programming table used to compute the cumulant function (see, e.g., Algorithm 2 in Ahmed et al. [2] for a detailed explanation).

A.2 Neighborhood graphs

Hypercube. On $\mathcal{Y}^d$, we use a family of neighborhood systems $\mathcal{N}_{\leq}^r$ parameterized by a Hamming distance radius $r \in [d-1]$. The graph is defined by:

$$\forall \mathbf{y}, \mathbf{y}' \in \mathcal{Y}^d : \mathbf{y}' \in \mathcal{N}_{\leq}^r(\mathbf{y}) \Leftrightarrow 1 \leq d_H(\mathbf{y}, \mathbf{y}') \leq r.$$

That is, two vertices are neighbors if their Hamming distance is at most r . This graph is regular, with degree $|\mathcal{N}_{\leq}^r(\mathbf{y})| = \sum_{i=1}^r \binom{d}{i}$. This graph is naturally connected, as any binary vector $\mathbf{y}'$ can be reached from any other binary vector $\mathbf{y}$ in $\|\mathbf{y}' - \mathbf{y}\|_1$ moves, by flipping each bit where $y'_i \neq y_i$, iteratively. Indeed, this trajectory consists in moves between vertices with Hamming distance equal to 1, and are therefore along edges of the neighborhood graph, regardless of the value of r .

We also use a slight variation on this family of neighborhood systems, the graphs $\mathcal{N}_{=}^r$, defined by:

$$\forall \mathbf{y}, \mathbf{y}' \in \mathcal{Y}^d : \mathbf{y}' \in \mathcal{N}_{=}^r(\mathbf{y}) \Leftrightarrow d_H(\mathbf{y}, \mathbf{y}') = r.$$

These graphs, on the contrary, are not always connected: indeed, if r is even, they contain two connected components (binary vectors with an even sum, and binary vectors with an odd sum). We only use such graphs when experimenting with neighborhood mixtures (see Algorithm 2), by aggregating them into a connected graph.

Top- κ polytope. On $\mathcal{Y}_\kappa^d$, we use a family of neighborhoods systems $\mathcal{N}^s$ parameterized by a number of “swaps” $s \in \llbracket 1, \min(\kappa, d - \kappa) \rrbracket$. The graph is defined by

$$\forall \mathbf{y}, \mathbf{y}' \in \mathcal{Y}_\kappa^d : \mathbf{y}' \in \mathcal{N}^s(\mathbf{y}) \Leftrightarrow d_H(\mathbf{y}, \mathbf{y}') = 2s.$$

That is, two vertices are neighbors if one can be reached from the other by performing s “swaps”, each swap corresponding to flipping a 1 to a 0 and vice-versa. This ensures that the resulting vector is still in $\mathcal{Y}_\kappa^d$. All s swaps must be performed on distinct components. The resulting graph is known as the *Generalized Johnson Graph* $J(d, \kappa, \kappa - s)$, or *Uniform Subset Graph* [12]. It is a regular graph, with degree $|\mathcal{N}^s(\mathbf{y})| = \binom{\kappa}{s} \binom{d-\kappa}{s}$. It is proved to be connected in Jones [24], except if $d = 2\kappa$ and $s = \kappa$ (in this case, it consists in $\frac{1}{2} \binom{d}{\kappa}$ disjoint edges).

When $s = 1$, the neighborhood graph is the Johnson Graph $J(d, \kappa)$, which coincides with the graph associated to the polytope $\text{conv}(\mathcal{Y}_\kappa^d) = \Delta_{d,\kappa}$ [43].

A.3 Convergence to exact gradients

In this section, we conduct experiments on the convergence of the MCMC estimators to the exact corresponding expectation (that is, convergence of the stochastic gradient estimator to the true gradient). The estimators are defined as

$$\hat{\mathbf{y}}_t(\boldsymbol{\theta}) = \mathbb{E}_{\pi_{\boldsymbol{\theta},t}}[Y] \approx \frac{1}{K - K_0} \sum_{k=K_0+1}^K \mathbf{y}^{(k)},$$

where $\mathbf{y}^{(k)}$ is the k -th iterate of Algorithm 1 with maximization direction $\boldsymbol{\theta}$, final temperature t , and K_0 is a number of burn-in (or warm-up) iterations. The obtained estimator is compared to the exact expectation by performing marginal inference as described in Appendix A.1 (with a closed-form formula in the case of $\mathcal{Y}^d$, and by dynamic programming in the case of $\mathcal{Y}_\kappa^d$).

Setup. For $T > K_0$, let $\tilde{\mathbb{E}}(\boldsymbol{\theta}, T) := \frac{1}{T - K_0} \sum_{k=K_0+1}^T \mathbf{y}^{(k)}$ be the stochastic estimate of the expectation at step T . We proceed by first randomly generating $\boldsymbol{\Theta} \in \mathbb{R}^{M \times d}$, with M being the number of instances, by sampling $\boldsymbol{\Theta}_{i,j} \sim \mathcal{N}(0, 1)$ independently. Then, we evaluate the impact of the following hyperparameters on the estimation of $\mathbb{E}_{\pi_{\boldsymbol{\Theta}_i,t}}[Y]$, for $i \in [M]$:

1. K_0 , the number of burn-in iterations,
2. t , the temperature parameter,
3. C , the number of parallel Markov chains.

Metric. The metric used is the squared Euclidean distance to the exact expectation, averaged on the M instances

$$\frac{1}{M} \sum_{i=1}^M \|\mathbb{E}_{\pi_{\boldsymbol{\Theta}_i,t}}[Y] - \tilde{\mathbb{E}}(\boldsymbol{\Theta}_i, T)\|_2^2,$$

which we measure for $T \in \llbracket K_0 + 1, K \rrbracket$.

Polytopes. For the hypercube $\mathcal{Y}^d$ and its neighborhood system $\mathcal{N}_\leq^r$, we use $d = 10$ and $r = 1$, which gives $|\mathcal{Y}^d| = 2^{10}$ and $|\mathcal{N}_\leq^r(\mathbf{y})| = 10$. For the top- κ polytope $\mathcal{Y}_\kappa^d$ and its neighborhood system $\mathcal{N}^s$, we use $d = 10$, $\kappa = 3$ and $s = 1$, which gives $|\mathcal{Y}_\kappa^d| = 120$ and $|\mathcal{N}^s(\mathbf{y})| = 30$. We also use a larger scale for both polytopes in order to highlight the varying impact of the temperature t depending on the combinatorial size of the problem, in the second experiment. For the large scale, we use $d = 1000$ and $r = 10$ for the hypercube, which give $|\mathcal{Y}^d| = 2^{1000} \approx 10^{301}$ and $|\mathcal{N}_\leq^r(\mathbf{y})| \approx 2.7 \times 10^{23}$, and we use $d = 1000$, $\kappa = 50$ and $s = 10$ for the top- κ polytope, which give $|\mathcal{Y}_\kappa^d| \approx 9.5 \times 10^{84}$ and $|\mathcal{N}^s(\mathbf{y})| \approx 1.6 \times 10^{33}$.

Hyperparameters. For each experiment, we use $K = 3000$. We average over $M = 1000$ problem instances for statistical significance. We use $K_0 = 0$, except for the first experiment, where it varies. We use a final temperature $t = 1$, except for the second experiment, where it varies. We use an initial temperature $t_0 = t = 1$ (leading to a constant temperature schedule), except for the first experiment, where it depends on K_0 . We use only one Markov chain and thus have $C = 1$, except for the third experiment, where it varies.

(1) Impact of burn-in. First, we evaluate the impact of K_0 , the number of burn-in iterations. We use a truncated geometric cooling schedule $t_k = \max(\gamma^k \cdot t_0, t)$ with $\gamma = 0.995$. The initial temperature t_0 is set to $1/(\gamma^{K_0})$, so that $\forall k \geq K_0 + 1, t_k = t = 1$. The results are gathered in Fig. 3.

(2) Impact of temperature. We then evaluate the impact of the final temperature t on the difficulty of the estimation task (different temperatures lead to different target expectations). The results for the small scale are gathered in Fig. 4, and the results for the large scale are gathered in Fig. 5.

(3) Impact of the number of parallel Markov chains. Finally, we evaluate the impact of the number of parallel Markov chains C on the estimation. The results are gathered in Fig. 6.

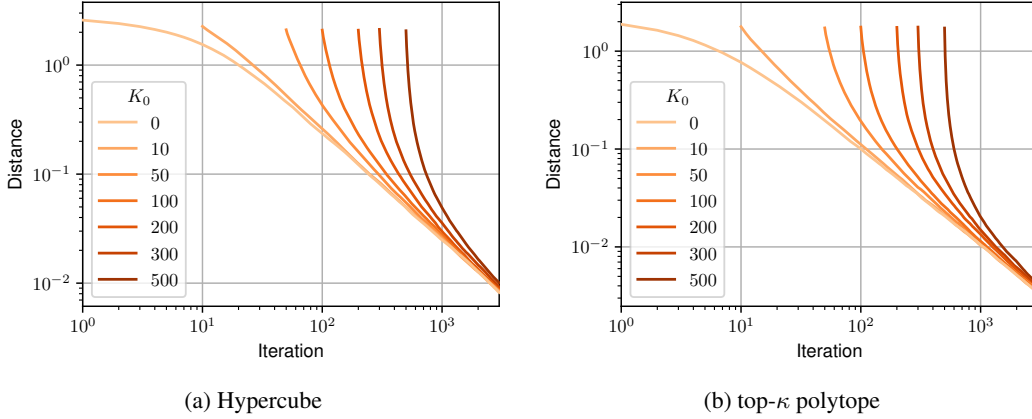


Figure 3: Convergence to exact expectation on the hypercube and the top- κ polytope, for varying number of burn-in iterations K_0 . We conclude that burn-in is not beneficial to the estimation, and taking $K_0 = 0$ is the best option.

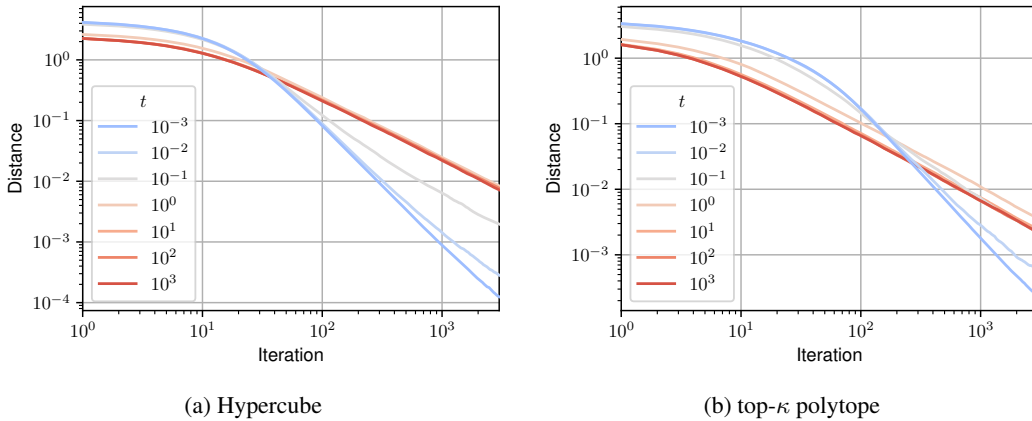


Figure 4: Convergence to exact expectation on the hypercube and the top- κ polytope, for varying final temperature t (small scale experiment). We conclude that lower temperatures facilitate the estimation.

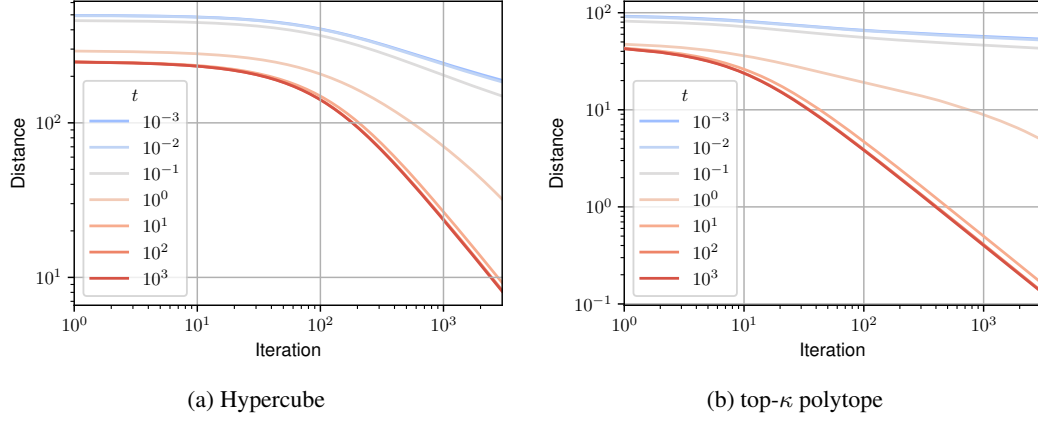


Figure 5: Convergence to exact expectation on the hypercube and the top- κ polytope, for varying final temperature t (large scale experiment). Contrary to the small scale case, larger temperatures are beneficial to the estimation when the solution set is combinatorially large.

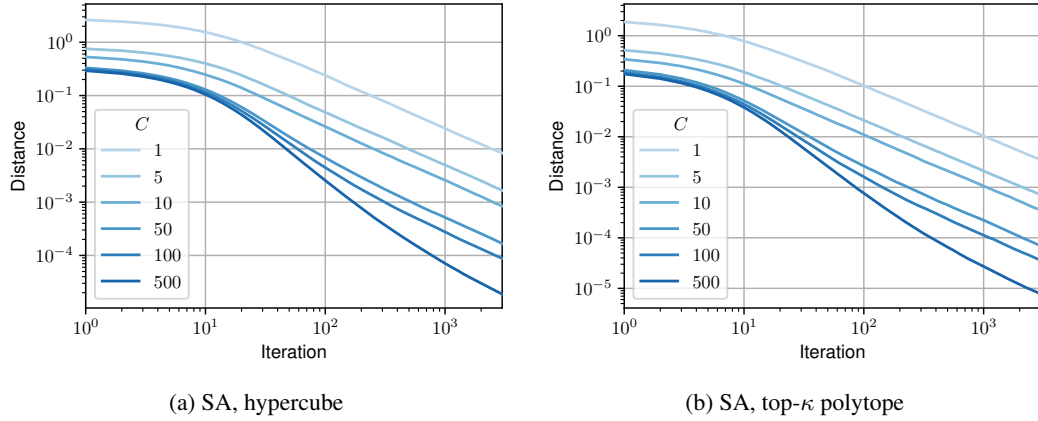


Figure 6: Convergence to exact expectation on the hypercube and the top- κ polytope, for varying number of parallel Markov chains C . Running 10 times more chains in parallel provides roughly the same benefit as extending each chain by 10 times more iterations, highlighting the advantage of massively parallelized estimation.

A.4 Convergence to exact parameters

In this section, we conduct experiments in the unsupervised setting described in Section 4.4. As a reminder, the empirical L_N and population L_{θ_0} Fenchel-Young losses are given by:

$$\begin{aligned}
 L_N(\theta; \mathbf{y}_1, \dots, \mathbf{y}_N) &:= \frac{1}{N} \sum_{i=1}^N \ell_t(\theta; \mathbf{y}_i) \\
 &= A_t(\theta) + \frac{1}{N} \sum_{i=1}^N \Omega_t(\mathbf{y}_i) - \langle \theta, \bar{Y}_N \rangle \\
 &= \ell_t(\theta; \bar{Y}_N) + C_1(Y),
 \end{aligned} \tag{8}$$

and

$$\begin{aligned}
 L_{\theta_0}(\theta) &:= \mathbb{E}_{(\mathbf{y}_i)_{i=1}^N \sim (\pi_{\theta_0, t})^{\otimes N}} [L_N(\theta; \mathbf{y}_1, \dots, \mathbf{y}_N)] \\
 &= A_t(\theta) + \mathbb{E}_{\pi_{\theta_0, t}} [\Omega_t(Y)] - \langle \theta, \hat{\mathbf{y}}_t(\theta_0) \rangle \\
 &= \ell_t(\theta; \hat{\mathbf{y}}_t(\theta_0)) + C_2(\theta_0),
 \end{aligned} \tag{9}$$

where the constants $C_1(Y) = \frac{1}{N} \sum_{i=1}^N \Omega_t(\mathbf{y}_i) - \Omega_t(\bar{Y}_N)$ and $C_2(\boldsymbol{\theta}_0) = \mathbb{E}_{\pi_{\boldsymbol{\theta}_0, t}} [\Omega_t(Y)] - \Omega_t(\hat{\mathbf{y}}_t(\boldsymbol{\theta}_0))$ do not depend on $\boldsymbol{\theta}$. As Jensen gaps, they are non-negative by convexity of Ω_t .

2D visualization. As an introductory example, we display stochastic gradient trajectories in Fig. 7. The parameter $\boldsymbol{\theta} \in \mathbb{R}^d$ is updated following Eq. (6) to minimize the population loss $L_{\boldsymbol{\theta}_0}$ defined in Eq. (9), with $\boldsymbol{\theta}_0 = (1/2, 1/2)$. The polytope used is the 2-dimensional hypercube $\mathcal{Y}^2$, with neighborhood graph $\mathcal{N}_1$ (neighbors are adjacent vertices of the square). We present trajectories obtained using MCMC-sampled gradients, comparing results from both 1 and 100 Markov chain iterations with Algorithm 1. For comparison, we include trajectories obtained using Monte Carlo-sampled (i.e., unbiased) gradients, using 1 and 100 samples.

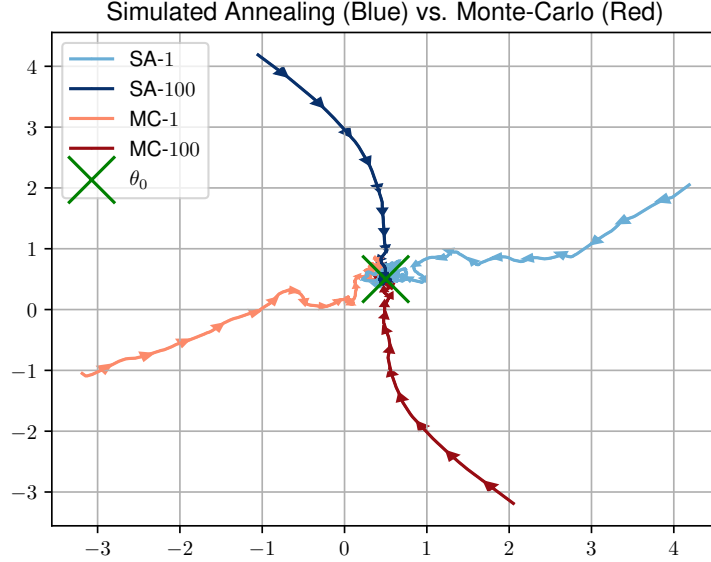


Figure 7: Comparison of stochastic gradient trajectories for a SA / M-H oracle on $\mathcal{Y}^2$ and unbiased stochastic gradients obtained via Monte Carlo sampling. Increasing the number of Markov chain iterations yields smoother trajectories, similar to the effect of using more Monte Carlo samples in the case of perturbation-based methods [6].

General setup. We proceed by first randomly generating true parameters $\boldsymbol{\Theta}_0 \in \mathbb{R}^{M \times d}$, with M being a number of problem instances we average on (in order to reduce noise in our observations), by sampling $\boldsymbol{\Theta}_{i,j} \sim \mathcal{N}(0, 1)$ independently. The goal is to learn each parameter vector $(\boldsymbol{\Theta}_0)_i \in \mathbb{R}^d, i \in [M]$, as M independent problems. The model is randomly initialized at $\hat{\boldsymbol{\Theta}}_0$, and updated with Adam [25] to minimize the loss. In order to better separate noise due to the optimization process and noise due to the sampling process, we use the population loss $L_{(\boldsymbol{\Theta}_0)_i}$ for general experiments, and use the empirical loss L_N only when focusing on the impact of the dataset size N . In this case, we create a dataset $Y \in \mathbb{R}^{M \times N \times d}$, with N being the number of samples, by sampling independently $Y_{i,j} \sim \pi_{(\boldsymbol{\Theta}_0)_i}, \forall i \in [M], \forall j \in [N]$.

We study the impact of the following hyperparameters on learning:

1. K , the number of Markov chain iterations,
2. C , the number of parallel Markov chains,
3. the initialization method used for the chains (either random, persistent, or data-based),
4. N , the number of samples in the dataset.

Metrics. The first metric used is the objective function actually minimized, i.e., the population loss, averaged on the M instances:

$$\frac{1}{M} \sum_{i=1}^M L_{(\Theta_0)_i}((\hat{\Theta}_n)_i),$$

where $(\hat{\Theta}_n)_i$ is the n -th iterate of the optimization process for the problem instance $i \in [M]$. We measure this loss for $n \in [n_{\max}]$, with $n_{\max}$ the total number of gradient iterations. For the fourth experiment, where we evaluate the impact of the number of samples N , we measure instead the empirical Fenchel-Young loss:

$$\frac{1}{M} \sum_{i=1}^M L_N((\hat{\Theta}_n)_i; Y_{i,1}, \dots, Y_{i,N})$$

In both cases, the best loss value that can be reached is positive but cannot be computed: it corresponds to the constants C_1 and C_2 in Eq. (8) and Eq. (9). Thus, we also provide "stretched" figures, where we plot the loss minus the best loss found during the optimization process.

The second metric used is the squared euclidean distance of the estimate to the true parameter, also averaged on the M instances:

$$\frac{1}{M} \sum_{i=1}^M \|(\Theta_0)_i - (\hat{\Theta}_n)_i\|_2^2.$$

As the top- κ polytope is of dimension $d - 1$, the model is only specified up to vectors orthogonal to the direction of the smallest affine subspace it spans. Thus, in this case, we measure instead:

$$\frac{1}{M} \sum_{i=1}^M \|P_D^\perp((\Theta_0)_i) - P_D^\perp((\hat{\Theta}_n)_i)\|_2^2,$$

where $P_D^\perp$ is the orthogonal projector on the hyperplane $D = \{\mathbf{x} \in \mathbb{R}^d : \langle \mathbf{1}, \mathbf{x} \rangle = 0\}$, which is the corresponding direction.

Polytopes. For the hypercube $\mathcal{Y}^d$ and its neighborhood system $\mathcal{N}_{\leq}^r$, we use $d = 10$ and $r = 1$, except in the fifth experiment, where we use a mixture of $\mathcal{N}_{\leq}^r$ neighborhoods (detailed below). For the top- κ polytope $\mathcal{Y}_{\kappa}^d$ and its neighborhood system $\mathcal{N}^s$, we use $d = 10$, $\kappa = 3$ and $s = 1$.

Hyperparameters. For each experiment, we perform 1000 gradient steps. We use $K_0 = 0$, final temperature $t = 1$ and initial temperature $t_0 = t = 1$ (leading to a constant temperature schedule). We use $K = 1000$ Markov chain iterations, except in the first experiment, where it varies. We use only one Markov chain and thus have $C = 1$, except for the second experiment, where it varies. We use a persistent initialization method for the Markov chains, except in the third experiment, where we compare the three different methods. For statistical significance, we average over $M = 100$ problem instances for each experiment, except in the third experiment, where we use $M = 1000$. We work in the limit case $N \rightarrow \infty$, except in the fourth experiment, where N varies.

(1) Impact of the length of Markov chains. First, we evaluate the impact of K , the number of inner iterations, i.e., the length of each Markov chain. The results are gathered in Fig. 8.

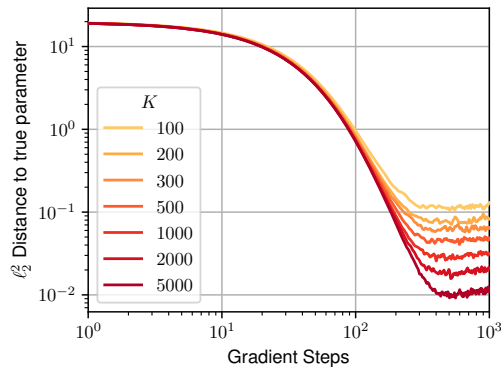
(2) Impact of the number of parallel Markov chains. We now evaluate the impact of the number of Markov chains C run in parallel to perform each gradient estimation on the learning process. The results are gathered in Fig. 9.

(3) Impact of the initialization method. Then, we evaluate the impact of the method to initialize each Markov chain used for gradient estimation. The persistent method consists in setting $\mathbf{y}^{(n+1,0)} = \mathbf{y}^{(n,K)}$, the data-based method consists in setting $\mathbf{y}^{(n+1,0)} = \mathbf{y}_i$ with $i \sim \mathcal{U}([N])$, and the random

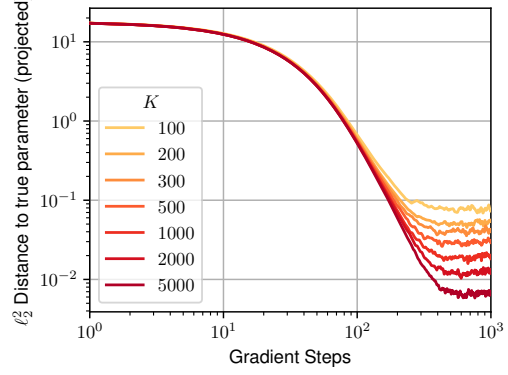
method consists in setting $\mathbf{y}^{(n+1,0)} \sim \mathcal{U}(\mathcal{Y})$ (see Appendix B.2 and Table 4 for a detailed explanation). The results are gathered in Fig. 10.

(4) Impact of the dataset size. We now evaluate the impact of the number of samples N from π_{θ_0} (i.e., the size of the dataset $(\mathbf{y}_i)_{i=1}^N$) on the estimation of the true parameter θ_0 . The results are gathered in Fig. 11.

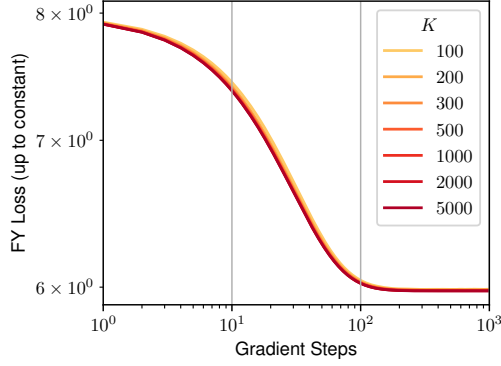
(5) Impact of neighborhood mixtures. Finally, we evaluate the impact of the use of neighborhood mixtures. To do so, we use mixtures $\{\mathcal{N}_{\underline{r}_s}^{r_s}\}_{s=1}^S$, once with $\{r_s\}_{s=1}^S = \{5\}$ opposed to $\{r_s\}_{s=1}^S = \{1, 5\}$, and once with $\{r_s\}_{s=1}^S = \{6\}$ (which gives a reducible Markov chain as 6 is even, so that the individual neighborhood graph $\mathcal{N}_{\underline{r}_s}^6$ is not connected, and has two connected components) opposed to $\{r_s\}_{s=1}^S = \{1, 2, 3, 6\}$. The results are gathered in Fig. 12.



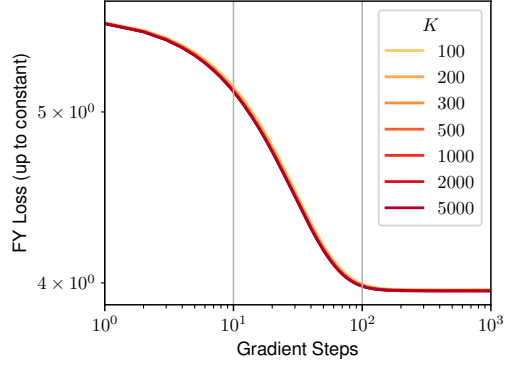
(a) Distance to true parameter, hypercube



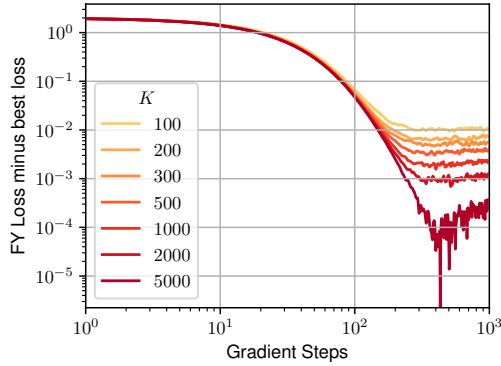
(b) Distance to true parameter, top- κ polytope



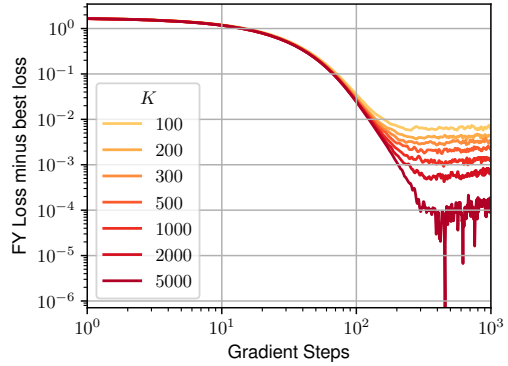
(c) FY loss (up to constant), hypercube



(d) FY loss (up to constant), top- κ polytope

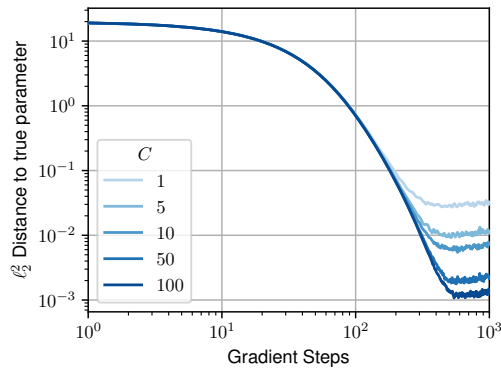


(e) FY loss minus best loss, hypercube

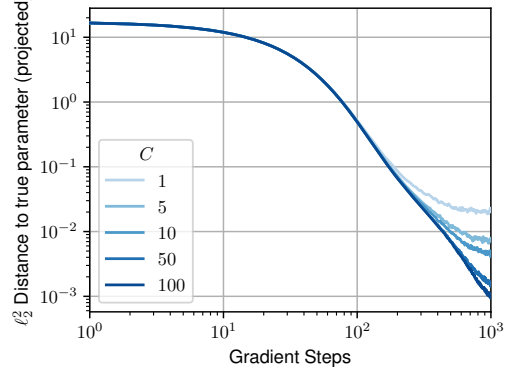


(f) FY loss minus best loss, top- κ polytope

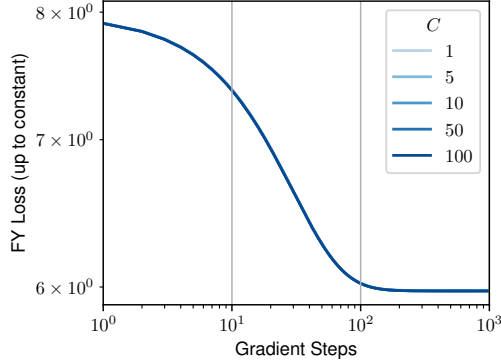
Figure 8: Convergence to the true parameter on the hypercube (left) and the top- κ polytope (right), for varying number of Markov chain iterations K . Longer chains improve learning.



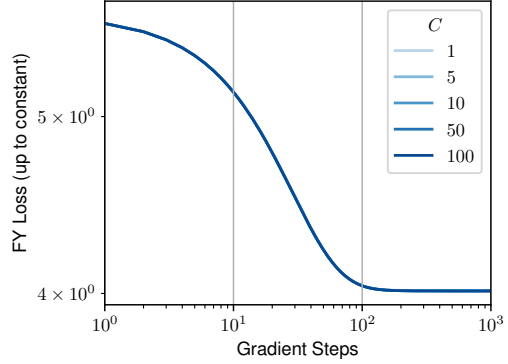
(a) Distance to true parameter, hypercube



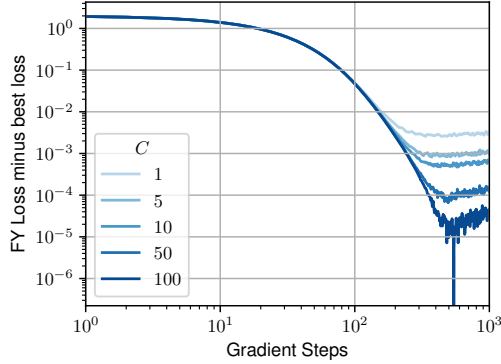
(b) Distance to true parameter, top- κ polytope



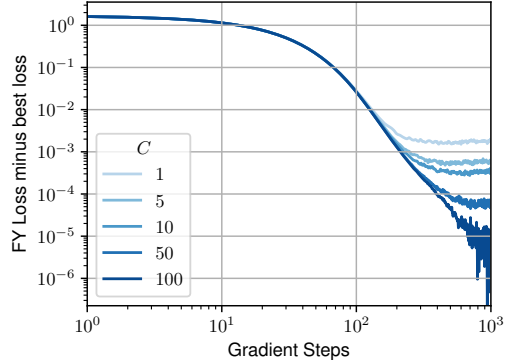
(c) FY loss (up to constant), hypercube



(d) FY loss (up to constant), top- κ polytope

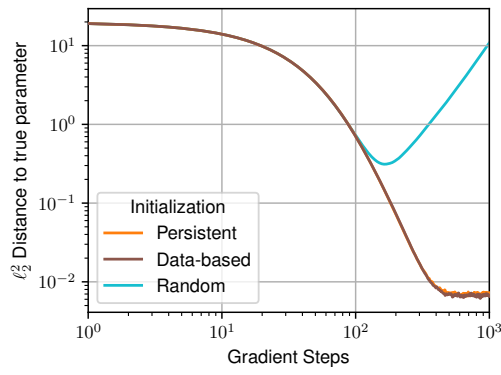


(e) FY loss minus best loss, hypercube

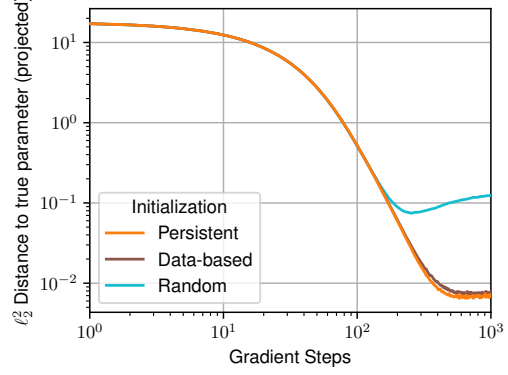


(f) FY loss minus best loss, top- κ polytope

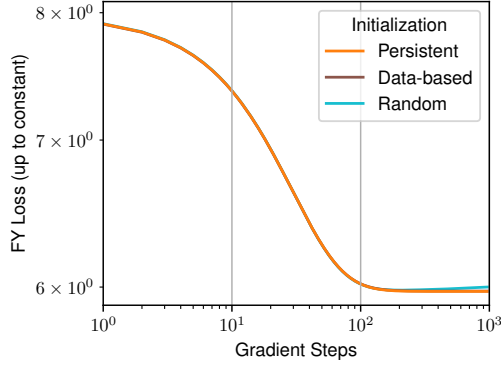
Figure 9: Convergence to the true parameter on the hypercube (left) and the top- κ polytope (right), for varying number of parallel Markov chains C . Adding Markov chains improves estimation.



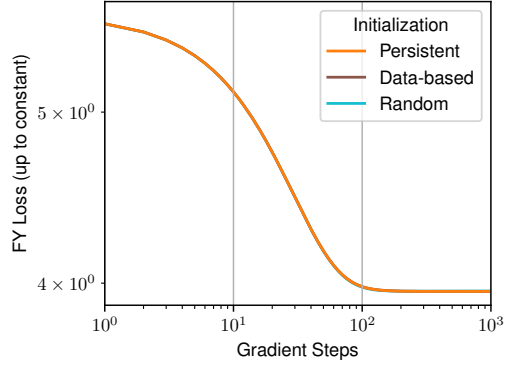
(a) Distance to true parameter, hypercube



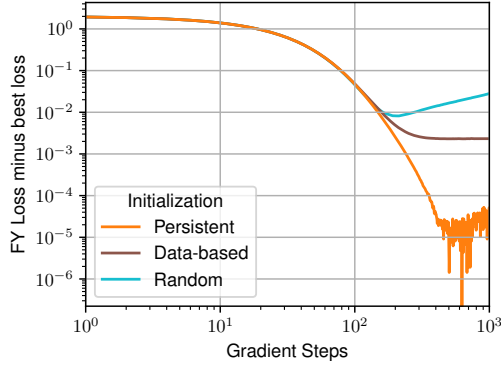
(b) Distance to true parameter, top- κ polytope



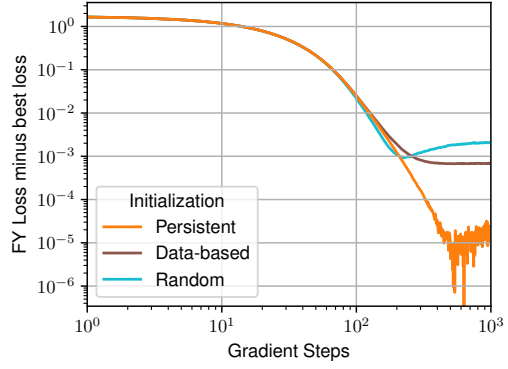
(c) FY loss (up to constant), hypercube



(d) FY loss (up to constant), top- κ polytope

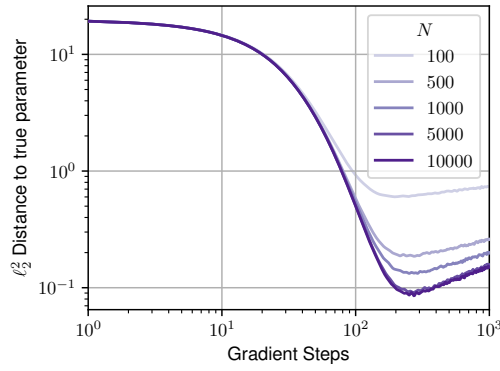


(e) FY loss minus best loss, hypercube

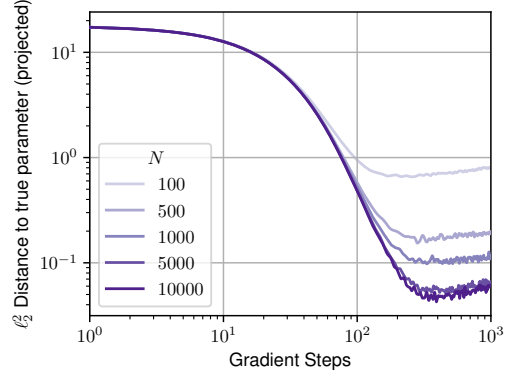


(f) FY loss minus best loss, top- κ polytope

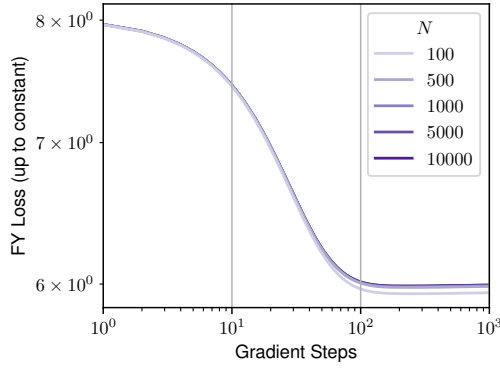
Figure 10: Convergence to the true parameter on the hypercube (left) and the top- κ polytope (right), for varying Markov chain initialization method. The persistent and data-based initialization methods significantly outperform the random initialization method.



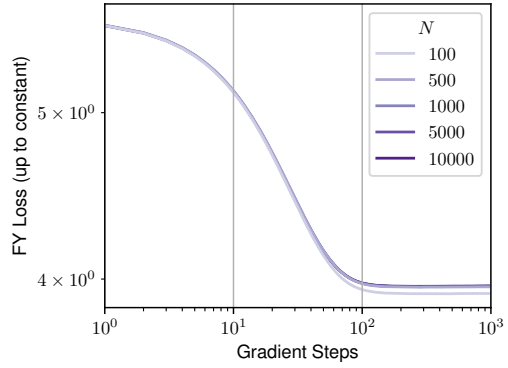
(a) Distance to true parameter



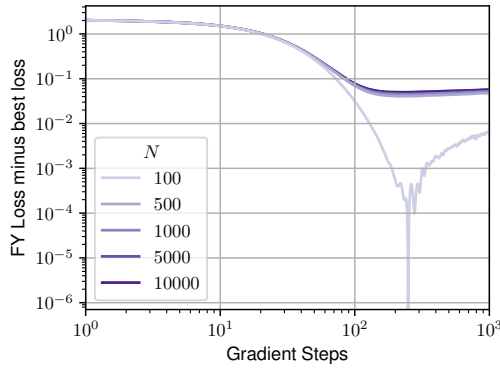
(b) Distance to true parameter



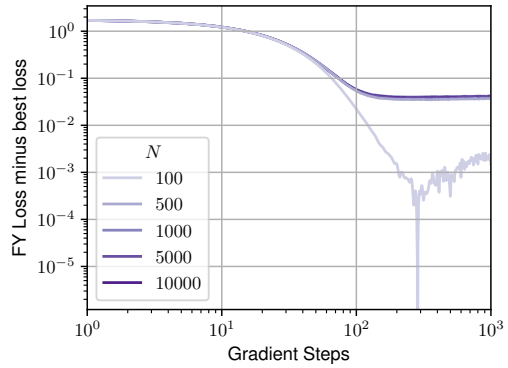
(c) FY loss (up to constant)



(d) Fenchel-Young loss (up to constant)

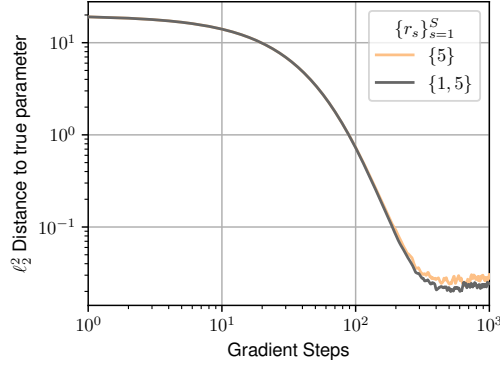


(e) FY loss minus best loss

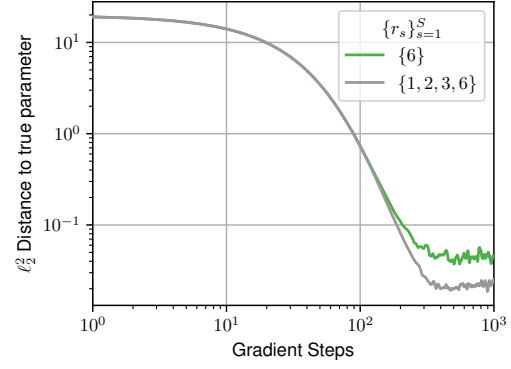


(f) Fenchel-Young loss minus best loss

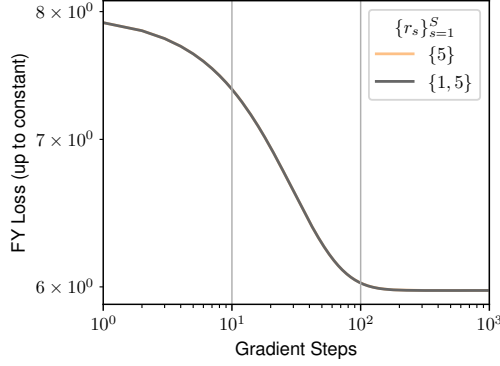
Figure 11: Convergence to the true parameter on the hypercube (left) and the top- κ polytope (right), for varying number of samples N in the dataset. As the dataset is different for each task, the empirical Fenchel-Young loss L_N , which is the minimized objective function (contrary to other experiments, where we minimize L_{θ_0}), also varies. Although empirical Fenchel-Young losses associated to smaller datasets appear easier to minimize, increasing the dataset size reduces the bias and thus the distance to θ_0 , as expected.



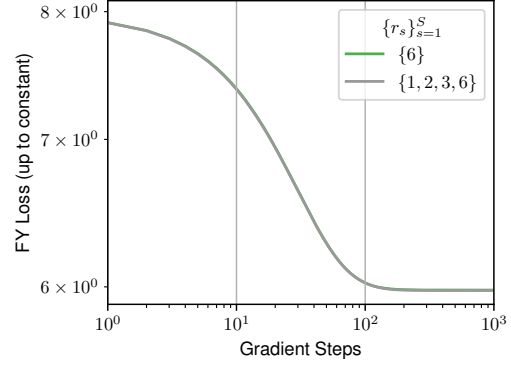
(a) Distance to true parameter, $r_s \in \{5\}$ or $\{1, 5\}$



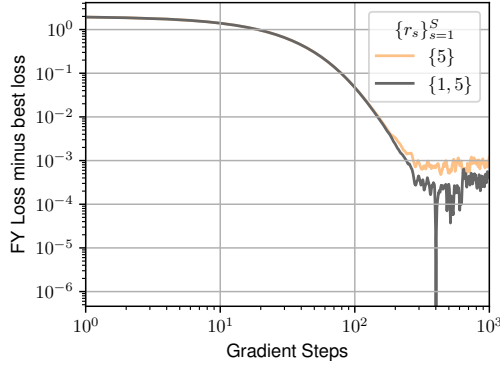
(b) Distance to true parameter, $r_s \in \{6\}$ or $\{1, 2, 3, 6\}$



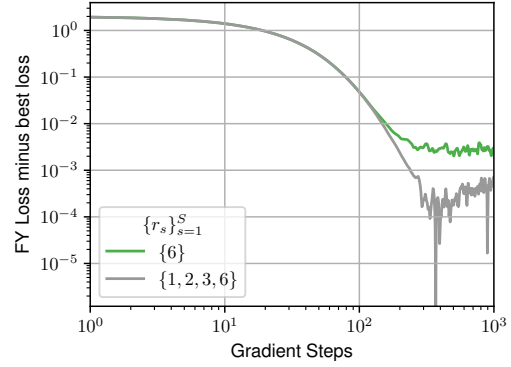
(c) FY loss (up to constant), $r_s \in \{5\}$ or $\{1, 5\}$



(d) FY loss (up to constant), $r_s \in \{6\}$ or $\{1, 2, 3, 6\}$



(e) FY loss minus best loss, $r_s \in \{5\}$ or $\{1, 5\}$



(f) FY loss minus best loss, $r_s \in \{6\}$ or $\{1, 2, 3, 6\}$

Figure 12: Convergence to the true parameter on the hypercube, with different mixtures of neighborhood systems $\{\mathcal{N}_{r_s}^S\}_{s=1}^S$: comparing $r_s \in \{5\}$ to $r_s \in \{1, 5\}$ (left), and comparing $r_s \in \{6\}$ to $r_s \in \{1, 2, 3, 6\}$ (right). Using more neighborhoods in the mixture improves learning.

B Additional material

B.1 Fenchel-Young loss for $K = 1$ in the unsupervised setting

This proposition is analogous to Proposition 3, but in the unsupervised setting, when using a data-based initialization method – i.e., the original CD initialization scheme, without persistent Markov chains. See Appendix B.2 for a detailed discussion about this.

Proposition 6. Let $\mathbf{p}_{\boldsymbol{\theta}, \bar{\mathbf{Y}}_N}^{(1)}$ denote the distribution of the first iterate of the Markov chain defined by the Markov transition kernel given in Eq. (3), with proposal distribution q and initialized by $\mathbf{y}^{(0)} = \mathbf{y}_i$, with $i \sim \mathcal{U}(\llbracket 1, N \rrbracket)$. There exists a dataset-dependent regularization $\Omega_{\bar{\mathbf{Y}}_N}$ with the following properties: $\Omega_{\bar{\mathbf{Y}}_N}$ is $tN / \sum_{i=1}^N \mathbb{E}_{q(\mathbf{y}_i, \cdot)} \|\mathbf{Y} - \mathbf{y}_i\|_2^2$ -strongly convex; it is such that:

$$\mathbb{E}_{\mathbf{p}_{\boldsymbol{\theta}, \bar{\mathbf{Y}}_N}^{(1)}} [\mathbf{Y}] = \underset{\boldsymbol{\mu} \in \text{conv}(\bigcup_{i=1}^N \{\mathcal{N}(\mathbf{y}_i) \cup \{\mathbf{y}_i\}\})}{\text{argmax}} \left\{ \langle \boldsymbol{\theta}, \boldsymbol{\mu} \rangle - \Omega_{\bar{\mathbf{Y}}_N}(\boldsymbol{\mu}) \right\};$$

and the Fenchel-Young loss $L_{\Omega_{\bar{\mathbf{Y}}_N}}$ generated by $\Omega_{\bar{\mathbf{Y}}_N}$ is $\frac{1}{N} \sum_{i=1}^N \mathbb{E}_{q(\mathbf{y}_i, \cdot)} \|\mathbf{Y} - \mathbf{y}_i\|_2^2 / t$ -smooth in its first argument, and such that $\nabla_{\boldsymbol{\theta}} L_{\Omega_{\bar{\mathbf{Y}}_N}}(\boldsymbol{\theta}; \mathbf{y}) = \mathbb{E}_{\mathbf{p}_{\boldsymbol{\theta}, \bar{\mathbf{Y}}_N}^{(1)}} [\mathbf{Y}] - \mathbf{y}$.

The proof is given in Appendix C.7.

B.2 Markov chain initialization

In contrastive divergence (CD) learning, the intractable expectation in the log-likelihood gradient is approximated by short-run MCMC, initialized at the data distribution [22] (using a Gibbs sampler in the setting of Restricted Boltzmann Machines).

Here, we note, at the n -th iteration of gradient descent:

$$\nabla_W L_N(\hat{W}_n) \approx \frac{1}{|B_n|} \sum_{i \in B_n} J_W g_{\hat{W}_n}(\mathbf{x}_i) \left(\frac{1}{K} \sum_{k=1}^K \mathbf{y}_i^{(n+1, k)} - \mathbf{y}_i \right),$$

for the supervised setting, with B_n being the mini-batch (or full batch) used at iteration n , $\mathbf{y}_i$ the ground-truth structure associated to $\mathbf{x}_i$ in the dataset, and $\mathbf{y}_i^{(n+1, k)}$ the k -th iterate of Algorithm 1, with maximization direction $g_{\hat{W}_n}(\mathbf{x}_i)$, and initialization point $\mathbf{y}_i^{(n+1, 0)}$. We also note:

$$\nabla_{\boldsymbol{\theta}} L_N(\hat{\boldsymbol{\theta}}_n) \approx \frac{1}{K} \sum_{k=1}^K \mathbf{y}^{(n+1, k)} - \bar{\mathbf{Y}}_N$$

for the unsupervised setting, with $\mathbf{y}^{(n+1, k)}$ being the k -th iterate of Algorithm 1, with maximization direction $\hat{\boldsymbol{\theta}}_n$, and initialization point $\mathbf{y}^{(n+1, 0)}$.

In CD learning of unconditional EBMs (i.e., in our unsupervised setting), the Markov Chain is initialized at the empirical data distribution [22, 11], as explained earlier. Persistent Contrastive Divergence (PCD) learning [50] modifies CD by maintaining a persistent Markov chain. Thus, instead of initializing the chain from the data distribution in each iteration, the chain continues from its last state in the previous iteration, by setting $\mathbf{y}^{(n+1, 0)} = \mathbf{y}^{(n, K)}$. This approach aims to provide a better approximation of the model distribution and to reduce the bias introduced by the initialization of the Markov chain in CD. These are two types of informative initialization methods, which aim at reducing the mixing times of the Markov Chains.

However, neither of these can be applied to the supervised (or conditional) setting, as observed in [38] in the context of conditional Restricted Boltzmann Machines (which are a type of EBMs). Indeed, on the one hand, PCD takes advantage of the fact that the parameter $\hat{\boldsymbol{\theta}}$ does not change too

much from one iteration to the next, so that a Markov Chain that has reached equilibrium on $\hat{\theta}_n$ is not far from equilibrium on $\hat{\theta}_{n+1}$. This does not hold in the supervised setting, as each x_i leads to a different $\hat{\theta}_i = g_{\hat{W}}(x_i)$. On the other hand, the data-based initialization method in CD would amount to initialize the chains at the empirical marginal data distribution on $\mathcal{Y}$, and would be irrelevant in a supervised setting, since the distribution we want each Markov Chain to approximate is conditioned on the input x_i .

An option is to use persistent chains if training for multiple epochs, and to initialize the Markov Chain associated to (x_i, y_i) for epoch j at the final state of the one associated to the same data point (x_i, y_i) at epoch $j - 1$. However, this method is relevant than PCD in the unsupervised setting, as $\hat{w}$ changes a lot more in a full epoch than $\hat{\theta}$ in just one gradient step in the unsupervised setting. It might be relevant, however, if each epoch consists in a single, full-batch gradient step. Nevertheless, it would require to store a significant number of states $y_i^{(n, K)}$ (one for each point in the dataset). The solution we propose, for both full-batch and mini-batch settings, is to initialize the chains at the empirical data distribution conditioned on the input x_i , which amounts to initialize them at the ground-truth y_i .

This discussion is summed up in Table 4.

Table 4: Possible Markov Chain Initialization Methods under each Learning Setting

Setting Init. Method	Unsupervised	Supervised, Batch	Supervised, Mini-Batch
Persistent	$y^{(n+1,0)} = y^{(n,K)}$	$y_i^{(n+1,0)} = y_i^{(n,K)}$	/
Data-Based	$y^{(n+1,0)} = y_j$, with $j \sim \mathcal{U}(\llbracket 1, N \rrbracket)$	$y_i^{(n+1,0)} = y_i$	$y_i^{(n+1,0)} = y_i$
Random	$y^{(n+1,0)} \sim \mathcal{U}(\mathcal{Y})$	$y_i^{(n+1,0)} \sim \mathcal{U}(\mathcal{Y})$	$y_i^{(n+1,0)} \sim \mathcal{U}(\mathcal{Y})$

Remark 1. *The use of uniform distributions on $\mathcal{Y}$ for the random initialization method can naturally be replaced by any other different prior distribution.*

B.3 Proposal distribution design for the DVRPTW

Original deterministic moves. The selected moves, designed for Local Search algorithms on vehicle routing problems (specifically for the PC-VRPTW for `serve request` and `remove request`), are given in Table 5.

All of these moves (except for `remove request`) involve selecting two clients i and j from the request set $\mathcal{R}^\omega$ (for example, the `relocate` move relocates client i after client j in the solution).

In the Local Search part of the PC-HGS algorithm from Vidal [51], they are implemented as deterministic functions used within a quadratic loop over clients, and are performed only if they improve the solution’s objective value. The search is narrowed down to client pairs (i, j) such that $d(i, j)$ is among the N_{prox} lowest values in $\{d(i, k) \mid k \in \mathcal{R}^\omega \setminus \{D, i\}\}$, where d is a problem-specific heuristic distance measure between clients, based on spatial features and time windows, and N_{prox} is a hyperparameter. These distances are independent from the chosen solution routes (they are computed once at the start of the algorithm, from the problem features), non-negative, and symmetric: $d(i, j) = d(j, i)$.

Randomization. In order to transform these deterministic moves into proposals, we first adapt the choice of clients i and j , by sampling i uniformly from $V_s^1(\mathbf{y})$, which contains the set of valid choices of client i for move s from solution $\mathbf{y}$. Then, we sample j from $V_s^2(\mathbf{y})[i] \setminus \{i\}$ using the following softmax distribution: $P_s(j \mid i) = \frac{\exp[-d(i, j)/\beta]}{\sum_{k \in V_s^2(\mathbf{y})[i] \setminus \{i\}} \exp[-d(i, k)/\beta]}$, where $\beta > 0$ is a neighborhood

Name	Description
relocate	removes request i from its route and re-inserts it before or after request j
relocate pair	removes pair of requests $(i, \text{next}(i))$ from their route and re-inserts them before or after request j
swap	exchanges the position of requests i and j in the solution
swap pair	exchanges the positions of the pairs $(i, \text{next}(i))$ and $(j, \text{next}(j))$ in the solution
2-opt	reverses the route segment between i and j
serve request	inserts currently undispached request i before or after request j
remove request	removes currently dispatched request i from the solution

Table 5: PC-VRPTW Local search moves

Move	$V_s^1(\mathbf{y})$	$V_s^2(\mathbf{y})[i]$
relocate	$\mathcal{D}(\mathbf{y}) \setminus \mathcal{D}_1(\mathbf{y})$	$\mathcal{D}(\mathbf{y})$
relocate pair	$\mathcal{D}(\mathbf{y}) \setminus \{\mathcal{D}_2(\mathbf{y}) \cup \mathcal{D}^{\text{last}}(\mathbf{y})\}$	$\mathcal{D}(\mathbf{y}) \setminus \{\mathcal{D}_2(\mathbf{y}) \cup \mathcal{D}^{\text{last}}(\mathbf{y}) \cup \{\text{next}(i)\}\}$
swap	$\mathcal{D}(\mathbf{y})$	$\mathcal{D}(\mathbf{y})$
swap pair	$\mathcal{D}(\mathbf{y}) \setminus \mathcal{D}^{\text{last}}(\mathbf{y})$	$\mathcal{D}(\mathbf{y}) \setminus \{\mathcal{D}^{\text{last}}(\mathbf{y}) \cup \{\text{prev}(i), \text{next}(i)\}\}$
2-opt	$\mathcal{D}(\mathbf{y}) \setminus \mathcal{D}_2(\mathbf{y})$	$\mathcal{D}(\mathbf{y}) \setminus \mathcal{D}_2(\mathbf{y})$
serve request	$\overline{\mathcal{D}}(\mathbf{y})$	$\mathcal{D}(\mathbf{y}) \cup \mathcal{I}_D(\mathbf{y})$
remove request	$\{\mathcal{D}(\mathbf{y}) \setminus \mathcal{D}_1(\mathbf{y})\} \cup \mathcal{I}_1(\mathbf{y})$	

Table 6: Sets of valid clients for each move. $\mathcal{D}(\mathbf{y})$ contains all dispatched clients in solution $\mathbf{y}$. $\mathcal{D}_1(\mathbf{y})$ contains all dispatched clients that are the only client in their route. $\mathcal{D}_2(\mathbf{y})$ contains all dispatched clients that are in a route with 2 clients or less. $\mathcal{D}^{\text{last}}(\mathbf{y})$ contains all dispatched clients that are the last of their route. $\overline{\mathcal{D}}(\mathbf{y})$ contains all non-dispatched clients. $\mathcal{I}_D(\mathbf{y})$ contains the depot of the first empty route, if it exists (all routes may be non-empty), or else is the empty set. $\mathcal{I}_1(\mathbf{y})$ contains the only client in the last non-empty route if it contains exactly one client, or else is the empty set.

sampling temperature. The set $V_s^2(\mathbf{y})[i]$ contains all valid choices of client j for move s from solution $\mathbf{y}$, and is precised along with $V_s^1(\mathbf{y})$ in Table 6. We normalize the distance measures inside the softmax, by dividing them by the maximum distance: $d(i, \cdot) \leftarrow d(i, \cdot) / \max_{k \in V_s^2(\mathbf{y})[i] \setminus \{i\}} d(i, k)$.

Neighborhood graph symmetrization. Then, we ensure that each individual neighborhood graph $\mathcal{N}_s$ is undirected. This is already the case for the moves `swap`, `swap pair` and `2-opt`, as they are actually involutions (applying the same move on the same couple (i, j) from $\mathbf{y}'$ will result in $\mathbf{y}$). However, this is obviously not the case for `serve request` and `remove request`. Indeed, if solution $\mathbf{y}'$ is obtained from $\mathbf{y}$ by removing a dispatched client (respectively serving an non-dispatched one), $\mathbf{y}$ cannot be obtained by removing another one (respectively, serving another one). To fix this, we merge these two moves into a single one. First, it evaluates which of the two moves are allowed (i.e., if they are such that $V_s^1(\mathbf{y}) \neq \emptyset$). Then, it samples one (the probability of selecting "remove" is chosen to be equal to the number of removable clients divided by the number of removable clients plus the number of servable clients) in the case where both are possible, or else simply performs the only move allowed. Thus, the corresponding neighborhood graph is undirected as it is always possible to perform the reverse operation (as when removing a client, it becomes unserved, thus allowing the `serve request` move from $\mathbf{y}'$, and vice-versa). We also allow the `serve request` move to insert a client after the depot of the first empty route, to allow the creation of new routes. In consequence, we allow the `remove request` move to remove the only client in the last non-empty route if it contains exactly one client (to maintain symmetry of the neighborhood graph).

For the `relocate` and `relocate pair` moves, the non-reversibility comes from the fact that they only relocate client i (or clients i and $\text{next}(i)$ in the pair case) after client j , so that if client i was the first in its route, relocating it back would be impossible (the depot, which is the start of the route, cannot be selected as j). Thus, we allow insertions before clients too, and add a random choice with probability $(\frac{1}{2}, \frac{1}{2})$ to determine if the relocated client(s) will be inserted before or after j . We also add this feature to the `serve request` move.

Correction ratio computation. Next, we implement the computation of the individual correction ratio $\tilde{\alpha}_s(\mathbf{y}, \mathbf{y}') = \frac{q_s(\mathbf{y}', \mathbf{y})}{q_s(\mathbf{y}, \mathbf{y}')}$ for each proposal q_s .

- In the case of `swap` and `2-opt`, we have $\tilde{\alpha}_s(\mathbf{y}, \mathbf{y}') = 1$. Indeed, let $\mathbf{y}'$ be the result of applying one of these moves s on $\mathbf{y}$ when sampling $i \in V_s^1(\mathbf{y})$ and $j \in V_s^2(\mathbf{y})[i] \setminus \{i\}$. We then have:

$$q_s(\mathbf{y}, \mathbf{y}') = \frac{1}{|V_s^1(\mathbf{y})|} \cdot \frac{\exp[-d(i, j)/\beta]}{\sum_{k \in V_s^2(\mathbf{y})[i] \setminus \{i\}} \exp[-d(i, k)/\beta]} + \frac{1}{|V_s^1(\mathbf{y})|} \cdot \frac{\exp[-d(j, i)/\beta]}{\sum_{k \in V_s^2(\mathbf{y})[j] \setminus \{j\}} \exp[-d(j, k)/\beta]},$$

where the first term accounts for the probability of selecting i then j and the second term accounts for that of selecting j then i (one can easily check that these two cases are the only way of sampling $\mathbf{y}'$ from $\mathbf{y}$). Then, noticing that we have $|V_s^1(\mathbf{y}')| = |V_s^1(\mathbf{y})|$, that these moves are involutions (selecting (i, j) or (j, i) from $\mathbf{y}'$ is also the only way to sample $\mathbf{y}$), and that we have the equalities $V_s^2(\mathbf{y})[i] = V_s^2(\mathbf{y}')[i]$ and $V_s^2(\mathbf{y})[j] = V_s^2(\mathbf{y}')[j]$, we actually have $q_s(\mathbf{y}', \mathbf{y}) = q_s(\mathbf{y}, \mathbf{y}')$.

- For `swap pair`, the same arguments hold (leading to the same form for q_s), except for the equalities $V_s^2(\mathbf{y})[i] = V_s^2(\mathbf{y}')[i]$ and $V_s^2(\mathbf{y})[j] = V_s^2(\mathbf{y}')[j]$. Thus, the ratio $\frac{q_s(\mathbf{y}', \mathbf{y})}{q_s(\mathbf{y}, \mathbf{y}')}$ must be computed explicitly, which has complexity $\mathcal{O}(\mathcal{R}^\omega)$.
- In the case of `relocate`, let j' denote $\text{next}(j)$ if the selected insertion type was "after", and $\text{prev}(j)$ if it was "before" – where $\text{next}(j) \in \mathcal{R}^\omega$ denotes the request following j in solution $\mathbf{y}$, i.e., the only index k such that $\mathbf{y}_{j,k} = 1$, and $\text{prev}(j)$ is the one preceding it, i.e., the only k such that $\mathbf{y}_{k,j} = 1$. We have:

$$q_s(\mathbf{y}, \mathbf{y}') = \frac{1}{2} \cdot \frac{1}{|V_s^1(\mathbf{y})|} \cdot \frac{\exp[-d(i, j)/\beta]}{\sum_{k \in V_s^2(\mathbf{y})[i] \setminus \{i\}} \exp[-d(i, k)/\beta]} + \frac{1}{2} \cdot \frac{1}{|V_s^1(\mathbf{y})|} \cdot \frac{\exp[-d(i, j')/\beta]}{\sum_{k \in V_s^2(\mathbf{y})[i] \setminus \{i\}} \exp[-d(i, k)/\beta]}$$

Indeed, if i was relocated *after* j , the same solution $\mathbf{y}'$ could have been obtained by relocating i *before* $j' = \text{next}(j)$. Similarly, if i was relocated *before* j , the same solution $\mathbf{y}'$ could have been obtained by relocating i *after* $j' = \text{prev}(j)$. For the reverse move probability, the way of obtaining $\mathbf{y}$ from $\mathbf{y}'$ is either to select $(i, \text{prev}(i))$ in the after-type insertion case, or $(i, \text{next}(i))$ in the before-type insertion case (where prev and next are taken w.r.t. $\mathbf{y}$, i.e., before applying the move). Thus, we have:

$$q_s(\mathbf{y}', \mathbf{y}) = \frac{1}{2} \cdot \frac{1}{|V_s^1(\mathbf{y}')|} \cdot \frac{\exp[-d(i, \text{prev}(i))/\beta]}{\sum_{k \in V_s^2(\mathbf{y}') [i] \setminus \{i\}} \exp[-d(i, k)/\beta]} + \frac{1}{2} \cdot \frac{1}{|V_s^1(\mathbf{y}')|} \cdot \frac{\exp[-d(i, \text{next}(i))/\beta]}{\sum_{k \in V_s^2(\mathbf{y}') [i] \setminus \{i\}} \exp[-d(i, k)/\beta]}.$$

- For the `relocate pair` move, the exact same reasoning and proposal probability form hold for the forward move, but we have for the reverse direction:

$$q_s(\mathbf{y}', \mathbf{y}) = \frac{1}{2} \cdot \frac{1}{|V_s^1(\mathbf{y}')|} \cdot \frac{\exp[-d(i, \text{prev}(i))/\beta]}{\sum_{k \in V_s^2(\mathbf{y}') [i] \setminus \{i\}} \exp[-d(i, k)/\beta]} + \frac{1}{2} \cdot \frac{1}{|V_s^1(\mathbf{y}')|} \cdot \frac{\exp[-d(i, \text{next}(\text{next}(i)))/\beta]}{\sum_{k \in V_s^2(\mathbf{y}') [i] \setminus \{i\}} \exp[-d(i, k)/\beta]},$$

as client $\text{next}(i)$ is also relocated.

- For the `serve request / remove request` move, we have the forward probability:

$$q_s(\mathbf{y}, \mathbf{y}') = \frac{|\{\mathcal{D}(\mathbf{y}) \setminus \mathcal{D}_1(\mathbf{y})\} \cup \mathcal{I}_1(\mathbf{y})|}{|\{\mathcal{D}(\mathbf{y}) \setminus \mathcal{D}_1(\mathbf{y})\} \cup \mathcal{I}_1(\mathbf{y})| + |\overline{\mathcal{D}}(\mathbf{y})|} \times \frac{1}{|\{\mathcal{D}(\mathbf{y}) \setminus \mathcal{D}_1(\mathbf{y})\} \cup \mathcal{I}_1(\mathbf{y})|} \\ = \frac{1}{|\{\mathcal{D}(\mathbf{y}) \setminus \mathcal{D}_1(\mathbf{y})\} \cup \mathcal{I}_1(\mathbf{y})| + |\overline{\mathcal{D}}(\mathbf{y})|}$$

if the chosen move is `remove request`. The expression corresponds to the composition of move choice sampling and uniform sampling over removable clients.

Still in the same case (`remove request` is chosen) and if the removed request i was in $\mathcal{I}_1(\mathbf{y})$ (i.e., was the only client in the last non-empty route if the latter contained exactly one client), we have the reverse move probability:

$$\begin{aligned} q_s(\mathbf{y}', \mathbf{y}) &= \frac{|\overline{\mathcal{D}}(\mathbf{y}')|}{|\{\mathcal{D}(\mathbf{y}') \setminus \mathcal{D}_1(\mathbf{y}')\} \cup \mathcal{I}_1(\mathbf{y}')| + |\overline{\mathcal{D}}(\mathbf{y}')|} \\ &\times \frac{\exp[-\bar{d}(i)/\beta]}{\exp[-\bar{d}(i)/\beta] + \sum_{k \in \mathcal{D}(\mathbf{y}')} \exp[-d(i, k)/\beta]} \\ &= \frac{|\overline{\mathcal{D}}(\mathbf{y})| + 1}{|\{\mathcal{D}(\mathbf{y}) \setminus \mathcal{D}_1(\mathbf{y})\} \cup \mathcal{I}_1(\mathbf{y})| + |\overline{\mathcal{D}}(\mathbf{y})|} \\ &\times \frac{\exp[-\bar{d}(i)/\beta]}{\exp[-\bar{d}(i)/\beta] + \sum_{\substack{k \in \mathcal{D}(\mathbf{y}) \\ k \neq i}} \exp[-d(i, k)/\beta]}. \end{aligned}$$

The expression corresponds to the composition of move choice sampling and softmax sampling of the depot of the first empty route (which was the route of the removed client i , so that $\mathcal{I}_D(\mathbf{y}') \neq \emptyset$ in this case). We use the average distance to dispatched clients $\bar{d}(i) := \frac{1}{|\mathcal{D}(\mathbf{y}')|} \sum_{k \in \mathcal{D}(\mathbf{y}')} d(i, k)$ as distance to the depot.

In the case where the removed request i was not in $\mathcal{I}_1(\mathbf{y})$, we have instead:

$$\begin{aligned} q_s(\mathbf{y}', \mathbf{y}) &= \frac{|\overline{\mathcal{D}}(\mathbf{y}')|}{|\{\mathcal{D}(\mathbf{y}') \setminus \mathcal{D}_1(\mathbf{y}')\} \cup \mathcal{I}_1(\mathbf{y}')| + |\overline{\mathcal{D}}(\mathbf{y}')|} \\ &\times \frac{\frac{1}{2} \cdot \exp[-d(i, \text{prev}(i))] + \frac{1}{2} \cdot \exp[-d(i, \text{next}(i))]}{\mathbf{1}_{\{\mathcal{I}_D(\mathbf{y}') \neq \emptyset\}} \cdot \exp[-\bar{d}(i)/\beta] + \sum_{k \in \mathcal{D}(\mathbf{y}')} \exp[-d(i, k)/\beta]} \\ &= \frac{|\overline{\mathcal{D}}(\mathbf{y})| + 1}{|\{\mathcal{D}(\mathbf{y}) \setminus \mathcal{D}_1(\mathbf{y})\} \cup \mathcal{I}_1(\mathbf{y})| + |\overline{\mathcal{D}}(\mathbf{y})|} \\ &\times \frac{\frac{1}{2} \cdot \exp[-d(i, \text{prev}(i))] + \frac{1}{2} \cdot \exp[-d(i, \text{next}(i))]}{\mathbf{1}_{\{\mathcal{I}_D(\mathbf{y}') \neq \emptyset\}} \cdot \exp[-\bar{d}(i)/\beta] + \sum_{\substack{k \in \mathcal{D}(\mathbf{y}) \\ k \neq i}} \exp[-d(i, k)/\beta]}. \end{aligned}$$

The right term corresponds to softmax sampling of the previous node with "after" insertion type (which has probability 1/2) and of the next node with "before" insertion type. The non-emptiness of $\mathcal{I}_D(\mathbf{y}')$ is not guaranteed anymore, as all routes might be non-empty (indeed, we did not create an empty one by removing i , as $i \in \mathcal{D}(\mathbf{y}) \setminus \mathcal{D}_1(\mathbf{y})$ in this case).

Similarly, if the chosen move is `serve request`, we have the forward probability:

$$\begin{aligned} q_s(\mathbf{y}, \mathbf{y}') &= \frac{|\overline{\mathcal{D}}(\mathbf{y})|}{|\{\mathcal{D}(\mathbf{y}) \setminus \mathcal{D}_1(\mathbf{y})\} \cup \mathcal{I}_1(\mathbf{y})| + |\overline{\mathcal{D}}(\mathbf{y})|} \\ &\times \frac{\frac{1}{2} \cdot \exp[-d(i, j)] + \frac{1}{2} \cdot \exp[-d(i, j')]}{\mathbf{1}_{\{\mathcal{I}_D(\mathbf{y}) \neq \emptyset\}} \cdot \exp[-\bar{d}(i)/\beta] + \sum_{k \in \mathcal{D}(\mathbf{y})} \exp[-d(i, k)/\beta]} \end{aligned}$$

if the selected insertion node j is not in $\mathcal{I}_D(\mathbf{y})$ (i.e., is not the depot of the first empty route in $\mathbf{y}$), where $j' = \text{prev}(j)$ if the insertion type selected was "before" (which has probability 1/2), and $j' = \text{next}(j)$ if it was "after".

We have instead the forward probability:

$$\begin{aligned} q_s(\mathbf{y}, \mathbf{y}') &= \frac{|\overline{\mathcal{D}}(\mathbf{y})|}{|\{\mathcal{D}(\mathbf{y}) \setminus \mathcal{D}_1(\mathbf{y})\} \cup \mathcal{I}_1(\mathbf{y})| + |\overline{\mathcal{D}}(\mathbf{y})|} \\ &\times \frac{\exp[-\bar{d}(i)/\beta]}{\exp[-\bar{d}(i)/\beta] + \sum_{k \in \mathcal{D}(\mathbf{y})} \exp[-d(i, k)/\beta]} \end{aligned}$$

if the selected insertion node j is in $\mathcal{I}_D(\mathbf{y})$ (i.e., is the depot of the first empty route in $\mathbf{y}$). In every case, we have the reverse move probability:

$$q_s(\mathbf{y}', \mathbf{y}) = \frac{1}{|\{\mathcal{D}(\mathbf{y}) \setminus \mathcal{D}_1(\mathbf{y})\} \cup \mathcal{I}_1(\mathbf{y})| + |\overline{\mathcal{D}}(\mathbf{y})|}.$$

In each case, we set $d(i, D) = +\infty$ to account for the fact that the depot can never be sampled during the process (except in the `serve request / remove request` move, where we allow the depot of the first empty route / last non-empty route to be selected, for which we use the average distance to other requests as explained earlier) – in fact, the distance measure from a client to the depot is not even defined in the original HGS implementation.

The second correction factor needed is $\frac{|Q(\mathbf{y})|}{|Q(\mathbf{y}')|}$ (see Algorithm 2). We compute it by checking if each move is allowed, i.e., if there exists at least one $i \in V_s^1(\mathbf{y})$ such that $V_s^2(\mathbf{y})[i] \setminus \{i\} \neq \emptyset$. This can be determined in $\mathcal{O}(\mathcal{R}^\omega)$ for each move.

B.4 Additional experimental details and results for Section 5

Model, features, dataset, hyperparameters, compute. Following Baty et al. [4], the differentiable ML model g_W is implemented as a sparse graph neural network. We use the same feature set (named *complete* feature set, and described in the Table 4 of their paper). We use the same training, validation, and testing datasets, which are created from 30, 15 and 25 problem instances respectively. The training set uses a sample size of 50 requests per wave, while the rest use 100. The solutions in the training dataset, i.e., the examples from the anticipative strategy f^* imitated by the model, are obtained by solving the corresponding offline VRPTWs using HGS [51] with a time limit of 3600 seconds. During evaluation, the PC-HGS solver $\tilde{g}$ is used with a constant time limit of 60 seconds for all models. We use Adam [25] together with the proposed stochastic gradient estimators, with a learning rate of $5 \cdot 10^{-3}$. Each training is performed using only a single CPU worker. For Fig. 2, we use a temperature $t = 10^2$. For Table 3, we use 1 Monte-Carlo sample for the perturbation-based method and 1 Markov chain for the proposed approach (in order to have a fair comparison: an equal number of oracle calls / equal compute).

Statistical significance. Each training is performed 50 times with the same parameters and different random seeds. Then, the learning curves are averaged, and plotted with a 95% confidence interval. For the results in Table 3, we report the performance of the best model iteration (selected with respect to the validation set) on the test set. This procedure is also averaged over 50 trainings, and reported with 95% confidence intervals.

Additional results. In Fig. 13, we report model performance for varying temperature t . Interestingly, lower temperatures perform better when using random initialization. In the ground-truth initialization setting, a sweet spot is found at $t = 10^2$, but lower temperatures do not particularly decrease performance.

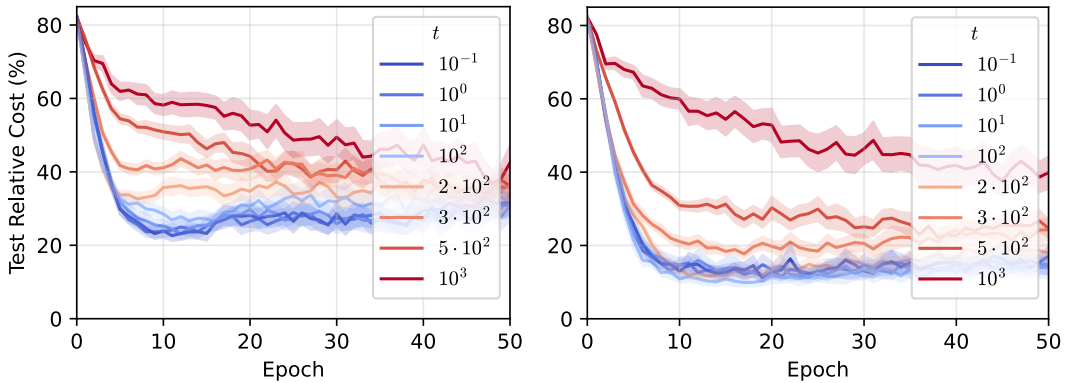


Figure 13: Test relative cost (%). **Left:** varying temperature t with random initialization. **Right:** varying temperature t with ground-truth initialization.

C Proofs

C.1 Proof of Eq. (4)

Proof. At fixed temperature $t_k = t$, the iterates of Algorithm 1 (MH case) follow a time-homogenous Markov chain, defined by the following transition kernel $P_{\theta,t}$:

$$P_{\theta,t}(\mathbf{y}, \mathbf{y}') = \begin{cases} q(\mathbf{y}, \mathbf{y}') \min \left[1, \frac{q(\mathbf{y}', \mathbf{y})}{q(\mathbf{y}, \mathbf{y}')} \exp \left(\frac{\langle \boldsymbol{\theta}, \mathbf{y}' \rangle + \varphi(\mathbf{y}') - \langle \boldsymbol{\theta}, \mathbf{y} \rangle - \varphi(\mathbf{y})}{t} \right) \right] & \text{if } \mathbf{y}' \in \mathcal{N}(\mathbf{y}), \\ 1 - \sum_{\mathbf{y}'' \in \mathcal{N}(\mathbf{y})} P_{\theta,t}(\mathbf{y}, \mathbf{y}'') & \text{if } \mathbf{y}' = \mathbf{y}, \\ 0 & \text{else.} \end{cases}$$

Irreducibility. As we assumed the neighborhood graph $G_{\mathcal{N}}$ to be connected and undirected, the Markov Chain is irreducible as we have $\forall \mathbf{y} \in \mathcal{Y}, \forall \mathbf{y}' \in \mathcal{N}(\mathbf{y}), P_{\theta,t}(\mathbf{y}, \mathbf{y}') > 0$.

Aperiodicity. For simplicity, we directly assumed aperiodicity in the main text. Here, we show that this is a mild condition, which is verified for instance if there is a solution $\mathbf{y} \in \mathcal{Y}$ such that $q(\mathbf{y}, \mathbf{y}) > 0$. Indeed, we then have:

$$\begin{aligned} P_{\theta,t}(\mathbf{y}, \mathbf{y}) &= 1 - \sum_{\mathbf{y}' \in \mathcal{N}(\mathbf{y})} P_{\theta,t}(\mathbf{y}, \mathbf{y}') \\ &= 1 - \sum_{\mathbf{y}' \in \mathcal{N}(\mathbf{y})} q(\mathbf{y}, \mathbf{y}') \min \left[1, \frac{q(\mathbf{y}', \mathbf{y})}{q(\mathbf{y}, \mathbf{y}')} \exp \left(\frac{\langle \boldsymbol{\theta}, \mathbf{y}' \rangle + \varphi(\mathbf{y}') - \langle \boldsymbol{\theta}, \mathbf{y} \rangle - \varphi(\mathbf{y})}{t} \right) \right] \\ &\geq 1 - \sum_{\mathbf{y}' \in \mathcal{N}(\mathbf{y})} q(\mathbf{y}, \mathbf{y}') \\ &\geq q(\mathbf{y}, \mathbf{y}) \\ &> 0. \end{aligned}$$

Thus, we have $P_{\theta,t}(\mathbf{y}, \mathbf{y}) > 0$, which implies that the chain is aperiodic. As an irreducible and aperiodic Markov Chain on a finite state space, it converges to its stationary distribution and the latter is unique [18]. Finally, one can easily check that the detailed balance equation is satisfied for $\pi_{\theta,t}$, i.e.:

$$\forall \mathbf{y}, \mathbf{y}' \in \mathcal{Y}, \pi_{\theta,t}(\mathbf{y}) P_{\theta,t}(\mathbf{y}, \mathbf{y}') = \pi_{\theta,t}(\mathbf{y}') P_{\theta,t}(\mathbf{y}', \mathbf{y}),$$

giving that $\pi_{\theta,t}$ is indeed the stationary distribution of the chain, which concludes the proof. $\square$

C.2 Proof of Proposition 1

Proof. Let $\boldsymbol{\theta} \in \mathbb{R}^d$ and $t > 0$. The fact that $\hat{\mathbf{y}}_t(\boldsymbol{\theta}) \in \text{relint}(\mathcal{C}) = \text{relint}(\text{conv}(\mathcal{Y}))$ follows directly from the fact that $\hat{\mathbf{y}}_t(\boldsymbol{\theta})$ is a convex combination of the elements of $\mathcal{Y}$ with positive coefficients, as $\forall \mathbf{y} \in \mathcal{Y}, \pi_{\theta,t}(\mathbf{y}) > 0$.

Low temperature limit. Let $\mathbf{y}^* := \arg\max_{\mathbf{y} \in \mathcal{Y}} \langle \boldsymbol{\theta}, \mathbf{y} \rangle + \varphi(\mathbf{y})$. The argmax is assumed to be single-valued. Let $\mathbf{y} \in \mathcal{Y} \setminus \{\mathbf{y}^*\}$. We have:

$$\begin{aligned} \pi_{\theta,t}(\mathbf{y}) &= \frac{\exp \left(\frac{\langle \boldsymbol{\theta}, \mathbf{y} \rangle + \varphi(\mathbf{y})}{t} \right)}{\sum_{\mathbf{y}' \in \mathcal{Y}} \exp \left(\frac{\langle \boldsymbol{\theta}, \mathbf{y}' \rangle + \varphi(\mathbf{y}')}{t} \right)} \\ &\leq \frac{\exp \left(\frac{\langle \boldsymbol{\theta}, \mathbf{y} \rangle + \varphi(\mathbf{y})}{t} \right)}{\exp \left(\frac{\langle \boldsymbol{\theta}, \mathbf{y}^* \rangle + \varphi(\mathbf{y}^*)}{t} \right)} \\ &\leq \exp \left(\frac{(\langle \boldsymbol{\theta}, \mathbf{y} \rangle + \varphi(\mathbf{y})) - (\langle \boldsymbol{\theta}, \mathbf{y}^* \rangle + \varphi(\mathbf{y}^*))}{t} \right) \\ &\xrightarrow[t \rightarrow 0^+]{} 0, \end{aligned}$$

as $\langle \theta, \mathbf{y} \rangle + \varphi(\mathbf{y}) < \langle \theta, \mathbf{y}^* \rangle + \varphi(\mathbf{y}^*)$ by definition of $\mathbf{y}^*$. Thus, we have:

$$\pi_{\theta,t}(\mathbf{y}^*) = 1 - \sum_{\mathbf{y} \in \mathcal{Y} \setminus \{\mathbf{y}^*\}} \pi_{\theta,t}(\mathbf{y}) \xrightarrow{t \rightarrow 0^+} 1.$$

Thus, the expectation of $\pi_{\theta,t}$ converges to $\mathbf{y}^*$. Naturally, if the argmax is not unique, the distribution converges to a uniform distribution on the maximizing structures.

High temperature limit. For all $\mathbf{y} \in \mathcal{Y}$, we have:

$$\begin{aligned} \pi_{\theta,t}(\mathbf{y}) &= \frac{\exp\left(\frac{\langle \theta, \mathbf{y} \rangle + \varphi(\mathbf{y})}{t}\right)}{\sum_{\mathbf{y}' \in \mathcal{Y}} \exp\left(\frac{\langle \theta, \mathbf{y}' \rangle + \varphi(\mathbf{y}')}{t}\right)} \\ &\xrightarrow{t \rightarrow +\infty} \frac{1}{|\mathcal{Y}|}, \end{aligned}$$

as $\exp(x/t) \xrightarrow{t \rightarrow +\infty} 1$ for all $x \in \mathbb{R}$. Thus, $\pi_{\theta,t}$ converges to the uniform distribution on $\mathcal{Y}$, and its expectation converges to the average of all structures.

Expression of the Jacobian. Let $A_t : \theta \mapsto t \cdot \log \sum_{\mathbf{y} \in \mathcal{Y}} \exp(\langle \theta, \mathbf{y} \rangle + \varphi(\mathbf{y}))$ be the cumulant function of the exponential family defined by $\pi_{\theta,t}$, scaled by t . One can easily check that we have $\nabla_{\theta} A_t(\theta) = \hat{\mathbf{y}}_t(\theta)$. Thus, we have $J_{\theta} \hat{\mathbf{y}}_t(\theta) = \nabla_{\theta}^2 A_t(\theta)$. However, we also have that the hessian matrix of the cumulant function $\theta \mapsto \frac{1}{t} A_t(\theta)$ is equal to the covariance matrix of the random vector $\frac{Y}{t}$ under $\pi_{\theta,t}$ [53]. Thus, we have:

$$\begin{aligned} J_{\theta} \hat{\mathbf{y}}_t(\theta) &= \nabla_{\theta}^2 A_t(\theta) \\ &= t \cdot \nabla_{\theta}^2 \left(\frac{1}{t} A_t(\theta) \right) \\ &= t \cdot \text{cov}_{\pi_{\theta,t}} \left[\frac{Y}{t} \right] \\ &= \frac{1}{t} \text{cov}_{\pi_{\theta,t}} [Y]. \end{aligned}$$

□

C.3 Proof of Proposition 2

Proof. Let $K_{\theta,t}$ be the Markov transition kernel associated to Algorithm 2, which can be written as:

$$K_{\theta,t}(\mathbf{y}, \mathbf{y}') = \begin{cases} \sum_{\substack{s \in Q(\mathbf{y}) \\ \text{s.t. } q_s(\mathbf{y}, \mathbf{y}') > 0}} \frac{1}{|Q(\mathbf{y})|} q_s(\mathbf{y}, \mathbf{y}') \min \left(1, \frac{|Q(\mathbf{y})|}{|Q(\mathbf{y}')|} \cdot \frac{q_s(\mathbf{y}', \mathbf{y}) \pi_{\theta,t}(\mathbf{y}')}{q_s(\mathbf{y}, \mathbf{y}') \pi_{\theta,t}(\mathbf{y})} \right) & \text{if } \mathbf{y}' \in \bar{\mathcal{N}}(\mathbf{y}), \\ 1 - \sum_{\mathbf{y}'' \in \bar{\mathcal{N}}(\mathbf{y})} K_{\theta,t}(\mathbf{y}, \mathbf{y}'') & \text{if } \mathbf{y}' = \mathbf{y}, \\ 0 & \text{else.} \end{cases}$$

As $\forall \mathbf{y} \in \mathcal{Y}, \forall \mathbf{y}' \in \bar{\mathcal{N}}(\mathbf{y}), K_{\theta,t}(\mathbf{y}, \mathbf{y}') > 0$, the irreducibility of the chain on $\mathcal{Y}$ is directly implied by the connectedness of $G_{\bar{\mathcal{N}}}$.

Thus, we only have to check that the detailed balance equation $\pi_{\theta,t}(\mathbf{y}) K_{\theta,t}(\mathbf{y}, \mathbf{y}') = \pi_{\theta,t}(\mathbf{y}') K_{\theta,t}(\mathbf{y}', \mathbf{y})$ is satisfied for all $\mathbf{y}' \in \bar{\mathcal{N}}(\mathbf{y})$. We have:

$$\pi_{\theta,t}(\mathbf{y}) K_{\theta,t}(\mathbf{y}, \mathbf{y}') = \sum_{\substack{s \in Q(\mathbf{y}) \\ \text{s.t. } q_s(\mathbf{y}, \mathbf{y}') > 0}} \left[\frac{q_s(\mathbf{y}, \mathbf{y}') \pi_{\theta,t}(\mathbf{y})}{|Q(\mathbf{y})|} \min \left(1, \frac{|Q(\mathbf{y})|}{|Q(\mathbf{y}')|} \cdot \frac{q_s(\mathbf{y}', \mathbf{y}) \pi_{\theta,t}(\mathbf{y}')}{q_s(\mathbf{y}, \mathbf{y}') \pi_{\theta,t}(\mathbf{y})} \right) \right].$$

The main point consists in noticing that the undirectedness assumption for each neighborhood graph $G_{\mathcal{N}_s}$ implies:

$$\{s \in Q(\mathbf{y}) : q_s(\mathbf{y}, \mathbf{y}') > 0\} = \{s \in Q(\mathbf{y}') : q_s(\mathbf{y}', \mathbf{y}) > 0\}.$$

Thus, a simple case analysis on how $|Q(\mathbf{y})| q_s(\mathbf{y}', \mathbf{y}) \pi_{\theta,t}(\mathbf{y}')$ and $|Q(\mathbf{y}')| q_s(\mathbf{y}, \mathbf{y}') \pi_{\theta,t}(\mathbf{y})$ compare allows us to observe that the expression of $\pi_{\theta,t}(\mathbf{y}) K_{\theta,t}(\mathbf{y}, \mathbf{y}')$ is symmetric in $\mathbf{y}$ and $\mathbf{y}'$, which concludes the proof. □

C.4 Proof of strict convexity

Proof. As A_t is a differentiable convex function on $\mathbb{R}^d$ (as the log-sum-exp of such functions), it is an essentially smooth closed proper convex function. Thus, it is such that

$$\text{relint}(\text{dom}((A_t)^*)) \subseteq \nabla A_t(\mathbb{R}^d) \subseteq \text{dom}((A_t)^*),$$

and we have that the restriction of $(A_t)^*$ to $\nabla A_t(\mathbb{R}^d)$ is strictly convex on every convex subset of $\nabla A_t(\mathbb{R}^d)$ (corollary 26.4.1 in Rockafellar [44]). As the range of the gradient of the cumulant function $\theta \mapsto A_t(\theta)/t$ is exactly the relative interior of the marginal polytope $\text{conv}(\{\mathbf{y}/t, \mathbf{y} \in \mathcal{Y}\})$ (see appendix B.1 in Wainwright and Jordan [53]), and $(A_t)^* =: \Omega_t$, we actually have that

$$\text{relint}(\text{dom}(\Omega_t)) \subseteq \text{relint}(\mathcal{C}) \subseteq \text{dom}(\Omega_t),$$

and that Ω_t is strictly convex on every convex subset of $\text{relint}(\mathcal{C})$, i.e., strictly convex on $\text{relint}(\mathcal{C})$ (as $\text{relint}(\mathcal{C})$ is itself convex).

As A_t is closed proper convex, it is equal to its biconjugate by the Fenchel-Moreau theorem. Thus, we have:

$$A_t(\theta) = \sup_{\mu \in \mathbb{R}^d} \{\langle \theta, \mu \rangle - (A_t)^*(\mu)\} = \sup_{\mu \in \mathbb{R}^d} \{\langle \theta, \mu \rangle - \Omega_t(\mu)\}.$$

Moreover, as $\nabla A_t(\mathbb{R}^d) = \text{relint}(\mathcal{C})$, we have $\|\nabla A_t(\theta)\| \leq R_C := \max_{\mu \in \mathcal{C}} \|\mu\|$, which gives $\text{dom}(\Omega_t) \subset B(\mathbf{0}, R_C)$. Thus we can actually write:

$$A_t(\theta) = \sup_{\mu \in B(\mathbf{0}, R_C)} \{\langle \theta, \mu \rangle - \Omega_t(\mu)\},$$

and now apply Danksin's theorem as $B(\mathbf{0}, R_C)$ is compact, which further gives:

$$\partial A_t(\theta) = \underset{\mu \in B(\mathbf{0}, R_C)}{\text{argmax}} \{\langle \theta, \mu \rangle - \Omega_t(\mu)\},$$

and the fact that A_t is differentiable gives that both sides are single-valued. Moreover, as $\nabla A_t(\mathbb{R}^d) = \text{relint}(\mathcal{C})$, we know that the right hand side is maximized in $\mathcal{C}$, and we can actually write:

$$\nabla A_t(\theta) = \underset{\mu \in \mathcal{C}}{\text{argmax}} \{\langle \theta, \mu \rangle - \Omega_t(\mu)\}.$$

We end this proof by noting that a simple calculation yields $\nabla A_t(\theta) = \mathbb{E}_{\pi_{\theta,t}}[Y] = \hat{\mathbf{y}}_t(\theta)$. The expression of $\nabla_{\theta} \ell_t(\theta; \mathbf{y})$ follows. $\square$

Remark 2. The proposed Fenchel-Young loss can also be obtained via distribution-space regularization. Let $s_{\theta} := (\langle \theta, \mathbf{y} \rangle + \varphi(\mathbf{y}))_{\mathbf{y} \in \mathcal{Y}} \in \mathbb{R}^{|\mathcal{Y}|}$ be a vector containing the score of all structures, and $L_{-tH} : \mathbb{R}^{|\mathcal{Y}|} \times \Delta^{|\mathcal{Y}|} \rightarrow \mathbb{R}$ be the Fenchel-Young loss generated by $-tH$, where H is the Shannon entropy. We have $\nabla_{s_{\theta}}(-tH)^*(s_{\theta}) = \pi_{\theta,t}$. The chain rule further gives $\nabla_{\theta}(-tH)^*(s_{\theta}) = \mathbb{E}_{\pi_{\theta,t}}[Y]$. Thus, we have $\nabla_{\theta} L_{-tH}(s_{\theta}; \mathbf{p}_{\mathbf{y}}) = \nabla_{\theta} \ell_t(\theta; \mathbf{y})$, where $\mathbf{p}_{\mathbf{y}}$ is the dirac distribution on $\mathbf{y}$. In the case where $\varphi \equiv 0$ and $t = 1$, we have $\Omega_t(\mu) = -(\max_{\mathbf{p} \in \Delta^{|\mathcal{Y}|}} H^s(\mathbf{p}) \text{ s.t. } \mathbb{E}_{\mathbf{p}}[Y] = \mu)$, with H^s the Shannon entropy [8], and ℓ_t is known as the CRF loss [29].

C.5 Proof of Proposition 4

Proof. The proof is exactly the proof of Proposition 4.1 in Berthet et al. [6], in which the setting is similar, and all the same arguments hold (we also have that π_{θ_0} is dense on $\mathcal{Y}$, giving $\bar{Y}_N \in \text{relint}(\mathcal{C})$ for N large enough). The only difference is the choice of regularization function, and we have to prove that it is also convex and smooth in our case. While the convexity of Ω_t is directly implied by its definition as a Fenchel conjugate, the fact that it is smooth is due to Theorem 26.3 in Rockafellar [44] and the essential strict convexity of A_t (which is itself closed proper convex). The latter relies on the fact that $\mathcal{C}$ is assumed to be of full-dimension (otherwise A_t would be linear when restricted to any affine subspace of direction equal to the subspace orthogonal to the direction of the smallest affine subspace spanned by $\mathcal{C}$), which in turn implies that A_t is strictly convex on $\mathbb{R}^d$. Thus, Proposition 4.1 in Berthet et al. [6] gives the asymptotic normality:

$$\sqrt{N}(\theta_N^* - \theta_0) \xrightarrow[N \rightarrow \infty]{\mathcal{D}} \mathcal{N}\left(\mathbf{0}, (\nabla_{\theta}^2 A_t(\theta_0))^{-1} \text{cov}_{\pi_{\theta_0,t}}[Y] (\nabla_{\theta}^2 A_t(\theta_0))^{-1}\right).$$

Moreover, we already derived $\nabla_{\theta}^2 A_t(\theta_0) = \frac{1}{t} \text{cov}_{\pi_{\theta_0,t}}[Y]$ in Appendix C.2, leading to the simplified asymptotic normality given in the proposition. $\square$

C.6 Proof of Proposition 5

Proof. The proof consists in bounding the convergence rate of the Markov chain $(\mathbf{y}^{(k)})_{k \in \mathbb{N}}$ (which has transition kernel $P_{\theta,t}$) for all θ , in order to apply Theorem 4.1 in Younes [54]. It is defined as the smallest constant λ_{θ} such that:

$$\exists A > 0 : \forall \mathbf{y} \in \mathcal{Y}, |\mathbb{P}(\mathbf{y}^{(k)} = \mathbf{y}) - \pi_{\theta,t}(\mathbf{y})| \leq A \lambda_{\theta}^k.$$

More precisely, we must find a constant D such that $\exists B > 0 : \lambda_{\theta} \leq 1 - B e^{-D \|\theta\|}$, in order to impose $K_{n+1} > \left\lceil 1 + a' \exp(2D \|\hat{\theta}_n\|) \right\rceil$.

A known result gives $\lambda_{\theta} \leq \rho(\theta)$ with $\rho(\theta) = \max_{\lambda \in S_{\theta} \setminus \{1\}} |\lambda|$ [32], where S_{θ} is the spectrum of the transition kernel $P_{\theta,t}$ (here, $1 - \rho(\theta)$ is known as the *spectral gap* of the Markov chain). To bound $\rho(\theta)$, we use the results of Ingrassia [23], which study the Markov chain with transition kernel $P'_{\theta,t}$, such that $P_{\theta,t} = \frac{1}{2} (I + P'_{\theta,t})$. It corresponds to the same algorithm, but with a proposal distribution q' defined as:

$$q'(\mathbf{y}, \mathbf{y}') = \begin{cases} \frac{1}{d^*} & \text{if } \mathbf{y}' \in \mathcal{N}(\mathbf{y}), \\ 1 - \frac{d(\mathbf{y})}{d^*} & \text{if } \mathbf{y}' = \mathbf{y}, \\ 0 & \text{else.} \end{cases}$$

As $P'_{\theta,t}$ is a row-stochastic matrix, Gershgorin's circle theorem gives that its spectrum is included in the complex unit disc. Moreover, as one can easily check that the associated Markov chain also has $\pi_{\theta,t}$ as stationary distribution, the detailed balance equation gives:

$$\forall \mathbf{y}, \mathbf{y}' \in \mathcal{Y}, \pi_{\theta,t}(\mathbf{y}) P'_{\theta,t}(\mathbf{y}, \mathbf{y}') = \pi_{\theta,t}(\mathbf{y}') P'_{\theta,t}(\mathbf{y}', \mathbf{y}),$$

which is equivalent to:

$$\forall \mathbf{y}, \mathbf{y}' \in \mathcal{Y}, \sqrt{\frac{\pi_{\theta,t}(\mathbf{y})}{\pi_{\theta,t}(\mathbf{y}')}} P'_{\theta,t}(\mathbf{y}, \mathbf{y}') = \sqrt{\frac{\pi_{\theta,t}(\mathbf{y}')}{\pi_{\theta,t}(\mathbf{y})}} P'_{\theta,t}(\mathbf{y}', \mathbf{y})$$

as $\pi_{\theta,t}$ has full support on $\mathcal{Y}$, which can be further written in matrix form as:

$$\Pi_{\theta}^{1/2} P'_{\theta,t} \Pi_{\theta}^{-1/2} = \Pi_{\theta}^{-1/2} P'_{\theta,t}{}^{\top} \Pi_{\theta}^{1/2},$$

where $\Pi_{\theta} = \text{diag}(\pi_{\theta,t})$. Thus, the matrix $\Pi_{\theta}^{1/2} P'_{\theta,t} \Pi_{\theta}^{-1/2}$ is symmetric, and the spectral theorem ensures its eigenvalues are real. As it is similar to the transition kernel $P'_{\theta,t}$ (with change of basis matrix $\Pi_{\theta}^{-1/2}$), they share the same spectrum S'_{θ} , and we have $S'_{\theta} \subset [-1, 1]$. Let us order S'_{θ} as $-1 \leq \lambda'_{\min} \leq \dots \leq \lambda'_2 \leq \lambda'_1 = 1$. As $P_{\theta,t} = \frac{1}{2} (I + P'_{\theta,t})$, we clearly have $\rho(\theta) = \frac{1+\lambda'_2}{2}$. Thus, we can use Theorem 4.1 of Ingrassia [23], which gives $\lambda'_2 \leq 1 - G \cdot Z(\theta) \exp(-m(\theta))$ (we keep their notations for Z and m , and add the dependency in θ for clarity), where G is a constant depending only on the graph $G_{\mathcal{N}}$, and with:

$$\begin{aligned}
Z(\boldsymbol{\theta}) &= \sum_{\mathbf{y} \in \mathcal{Y}} \exp \left(\frac{\langle \boldsymbol{\theta}, \mathbf{y} \rangle + \varphi(\mathbf{y})}{t} - \max_{\mathbf{y}' \in \mathcal{Y}} \left[\frac{\langle \boldsymbol{\theta}, \mathbf{y}' \rangle + \varphi(\mathbf{y}')}{t} \right] \right) \\
&\geq |\mathcal{Y}| \exp \left(\frac{1}{t} \left[\min_{\mathbf{y} \in \mathcal{Y}} \langle \boldsymbol{\theta}, \mathbf{y} \rangle + \min_{\mathbf{y} \in \mathcal{Y}} \varphi(\mathbf{y}) - \max_{\mathbf{y}' \in \mathcal{Y}} \langle \boldsymbol{\theta}, \mathbf{y}' \rangle - \max_{\mathbf{y}' \in \mathcal{Y}} \varphi(\mathbf{y}') \right] \right) \\
&\geq |\mathcal{Y}| \exp \left(-\frac{2R_{\mathcal{C}}}{t} \|\boldsymbol{\theta}\| - \frac{2R_{\varphi}}{t} \right),
\end{aligned}$$

and:

$$\begin{aligned}
m(\boldsymbol{\theta}) &\leq \max_{\mathbf{y} \in \mathcal{Y}} \left\{ \max_{\mathbf{y}' \in \mathcal{Y}} \left[\frac{\langle \boldsymbol{\theta}, \mathbf{y}' \rangle + \varphi(\mathbf{y}')}{t} \right] - \frac{\langle \boldsymbol{\theta}, \mathbf{y} \rangle + \varphi(\mathbf{y})}{t} \right\} - 2 \min_{\mathbf{y} \in \mathcal{Y}} \left\{ \max_{\mathbf{y}' \in \mathcal{Y}} \left[\frac{\langle \boldsymbol{\theta}, \mathbf{y}' \rangle + \varphi(\mathbf{y}')}{t} \right] - \frac{\langle \boldsymbol{\theta}, \mathbf{y} \rangle + \varphi(\mathbf{y})}{t} \right\} \\
&= \max_{\mathbf{y}' \in \mathcal{Y}} \left[\frac{\langle \boldsymbol{\theta}, \mathbf{y}' \rangle + \varphi(\mathbf{y}')}{t} \right] - \min_{\mathbf{y} \in \mathcal{Y}} \left[\frac{\langle \boldsymbol{\theta}, \mathbf{y} \rangle + \varphi(\mathbf{y})}{t} \right] \\
&\leq \frac{1}{t} \left(\max_{\mathbf{y}' \in \mathcal{Y}} \langle \boldsymbol{\theta}, \mathbf{y}' \rangle + \max_{\mathbf{y}' \in \mathcal{Y}} \varphi(\mathbf{y}') - \min_{\mathbf{y} \in \mathcal{Y}} \langle \boldsymbol{\theta}, \mathbf{y} \rangle - \min_{\mathbf{y} \in \mathcal{Y}} \varphi(\mathbf{y}) \right) \\
&\leq \frac{2R_{\mathcal{C}}}{t} \|\boldsymbol{\theta}\| + \frac{2R_{\varphi}}{t},
\end{aligned}$$

where $R_{\mathcal{C}} = \max_{\mathbf{y} \in \mathcal{Y}} \|\mathbf{y}\|$ and $R_{\varphi} = \max_{\mathbf{y} \in \mathcal{Y}} |\varphi(\mathbf{y})|$. Thus, we have:

$$\lambda'_2 \leq 1 - G|\mathcal{Y}| \exp \left(-\frac{4R_{\varphi}}{t} \right) \exp \left(-\frac{4R_{\mathcal{C}}}{t} \|\boldsymbol{\theta}\| \right),$$

and finally:

$$\lambda_{\boldsymbol{\theta}} \leq 1 - \frac{G|\mathcal{Y}| \exp \left(-\frac{4R_{\varphi}}{t} \right)}{2} \exp \left(-\frac{4R_{\mathcal{C}}}{t} \|\boldsymbol{\theta}\| \right),$$

so taking $D = 4R_{\mathcal{C}}/t$ concludes the proof. □

Remark 3. The stationary distribution in Ingrassia [23] is defined as proportional to $\exp(-H(\mathbf{y}))$, with the assumption that the function H is such that $\min_{\mathbf{y} \in \mathcal{Y}} H(\mathbf{y}) = 0$. Thus, we apply their results with

$$H(\mathbf{y}) := \max_{\mathbf{y}' \in \mathcal{Y}} \left[\frac{\langle \boldsymbol{\theta}, \mathbf{y}' \rangle + \varphi(\mathbf{y}')}{t} \right] - \frac{\langle \boldsymbol{\theta}, \mathbf{y} \rangle + \varphi(\mathbf{y})}{t}$$

(which gives correct distribution $\pi_{\boldsymbol{\theta}, t}$ and respects this assumption), hence the obtained forms for $Z(\boldsymbol{\theta})$ and the upper bound on $m(\boldsymbol{\theta})$.

C.7 Proofs of Proposition 3 and Proposition 6

Proposition 3. The distribution of the first iterate of the Markov chain with transition kernel defined in Eq. (3) and initialized at the ground-truth structure $\mathbf{y}$ is given by:

$$\begin{aligned}
(p_{\boldsymbol{\theta}, \mathbf{y}}^{(1)})(\mathbf{y}') &= P_{\boldsymbol{\theta}, t}(\mathbf{y}, \mathbf{y}') \\
&= \begin{cases} q(\mathbf{y}, \mathbf{y}') \min \left[1, \frac{q(\mathbf{y}', \mathbf{y})}{q(\mathbf{y}, \mathbf{y}')} \exp \left(\frac{[\langle \boldsymbol{\theta}, \mathbf{y}' - \mathbf{y} \rangle + \varphi(\mathbf{y}') - \varphi(\mathbf{y})]}{t} \right) \right] & \text{if } \mathbf{y}' \in \mathcal{N}(\mathbf{y}), \\ 1 - \sum_{\mathbf{y}'' \in \mathcal{N}(\mathbf{y})} (p_{\boldsymbol{\theta}, \mathbf{y}}^{(1)})(\mathbf{y}'') & \text{if } \mathbf{y}' = \mathbf{y}, \\ 0 & \text{else.} \end{cases}
\end{aligned}$$

Let $\alpha_{\mathbf{y}}(\boldsymbol{\theta}, \mathbf{y}') := \frac{q(\mathbf{y}', \mathbf{y})}{q(\mathbf{y}, \mathbf{y}')} \exp([\langle \boldsymbol{\theta}, \mathbf{y}' - \mathbf{y} \rangle + \varphi(\mathbf{y}') - \varphi(\mathbf{y})] / t)$. Define also the following sets:

$$\mathcal{N}_{\mathbf{y}}^-(\boldsymbol{\theta}) = \{\mathbf{y}' \in \mathcal{N}(\mathbf{y}) \mid \alpha_{\mathbf{y}}(\boldsymbol{\theta}, \mathbf{y}') \leq 1\}, \quad \mathcal{N}_{\mathbf{y}}^+(\boldsymbol{\theta}) = \{\mathbf{y}' \in \mathcal{N}(\mathbf{y}) \mid \alpha_{\mathbf{y}}(\boldsymbol{\theta}, \mathbf{y}') > 1\}.$$

The expectation of the first iterate is then given by:

$$\begin{aligned} \mathbb{E}_{\mathbf{p}_{\boldsymbol{\theta}, \mathbf{y}}^{(1)}}[Y] &= \sum_{\mathbf{y}' \in \mathcal{N}(\mathbf{y})} (\mathbf{p}_{\boldsymbol{\theta}, \mathbf{y}}^{(1)})(\mathbf{y}') \cdot \mathbf{y}' + \left(1 - \sum_{\mathbf{y}'' \in \mathcal{N}(\mathbf{y})} (\mathbf{p}_{\boldsymbol{\theta}, \mathbf{y}}^{(1)})(\mathbf{y}'')\right) \cdot \mathbf{y} \\ &= \mathbf{y} + \sum_{\mathbf{y}' \in \mathcal{N}(\mathbf{y})} (\mathbf{p}_{\boldsymbol{\theta}, \mathbf{y}}^{(1)})(\mathbf{y}') \cdot (\mathbf{y}' - \mathbf{y}) \\ &= \mathbf{y} + \sum_{\mathbf{y}' \in \mathcal{N}_{\mathbf{y}}^-(\boldsymbol{\theta})} q(\mathbf{y}', \mathbf{y}) \exp([\langle \boldsymbol{\theta}, \mathbf{y}' - \mathbf{y} \rangle + \varphi(\mathbf{y}') - \varphi(\mathbf{y})] / t) \cdot (\mathbf{y}' - \mathbf{y}) + \sum_{\mathbf{y}' \in \mathcal{N}_{\mathbf{y}}^+(\boldsymbol{\theta})} q(\mathbf{y}, \mathbf{y}') \cdot (\mathbf{y}' - \mathbf{y}). \end{aligned}$$

Let now $f_{\mathbf{y}} : \mathbb{R}^d \times \mathcal{N}(\mathbf{y}) \rightarrow \mathbb{R}$ be defined as:

$$f_{\mathbf{y}} : (\boldsymbol{\theta}; \mathbf{y}') \mapsto \begin{cases} t \cdot q(\mathbf{y}', \mathbf{y}) \exp([\langle \boldsymbol{\theta}, \mathbf{y}' - \mathbf{y} \rangle + \varphi(\mathbf{y}') - \varphi(\mathbf{y})] / t) & \text{if } \alpha_{\mathbf{y}}(\boldsymbol{\theta}, \mathbf{y}') \leq 1, \\ t \cdot q(\mathbf{y}, \mathbf{y}') \left([\langle \boldsymbol{\theta}, \mathbf{y}' - \mathbf{y} \rangle + \varphi(\mathbf{y}') - \varphi(\mathbf{y})] / t + 1 - \log \frac{q(\mathbf{y}, \mathbf{y}')}{q(\mathbf{y}', \mathbf{y})}\right) & \text{if } \alpha_{\mathbf{y}}(\boldsymbol{\theta}, \mathbf{y}') > 1. \end{cases}$$

Let $F_{\mathbf{y}} : \boldsymbol{\theta} \mapsto \langle \boldsymbol{\theta}, \mathbf{y} \rangle + \sum_{\mathbf{y}' \in \mathcal{N}(\mathbf{y})} f_{\mathbf{y}}(\boldsymbol{\theta}; \mathbf{y}')$. We define the target-dependent regularization function $\Omega_{\mathbf{y}}$ and the corresponding Fenchel-Young loss as:

$$\Omega_{\mathbf{y}} : \boldsymbol{\mu} \mapsto (F_{\mathbf{y}})^*(\boldsymbol{\mu}), \quad L_{\Omega_{\mathbf{y}}}(\boldsymbol{\theta}; \mathbf{y}) := (\Omega_{\mathbf{y}})^*(\boldsymbol{\theta}) + \Omega_{\mathbf{y}}(\mathbf{y}) - \langle \boldsymbol{\theta}, \mathbf{y} \rangle.$$

- $\Omega_{\mathbf{y}}$ is $t/\mathbb{E}_{q(\mathbf{y}, \cdot)}\|Y - \mathbf{y}\|_2^2$ -strongly convex:

One can easily check that $f_{\mathbf{y}}(\cdot; \mathbf{y}')$ is continuous for all $\mathbf{y}' \in \mathcal{N}(\mathbf{y})$, as it is defined piecewise as continuous functions that match on the junction affine hyperplane defined by:

$$\{\boldsymbol{\theta} \in \mathbb{R}^d \mid \alpha_{\mathbf{y}}(\boldsymbol{\theta}; \mathbf{y}') = 1\} = \left\{ \boldsymbol{\theta} \in \mathbb{R}^d \mid \langle \boldsymbol{\theta}, \mathbf{y}' - \mathbf{y} \rangle = t \log \frac{q(\mathbf{y}, \mathbf{y}')}{q(\mathbf{y}', \mathbf{y})} + \varphi(\mathbf{y}) - \varphi(\mathbf{y}') \right\}.$$

Moreover, we have that $f_{\mathbf{y}}(\cdot; \mathbf{y}')$ is actually differentiable everywhere as its gradient can be continuously extended to the junction affine hyperplane with constant value equal to $q(\mathbf{y}, \mathbf{y}')(\mathbf{y}' - \mathbf{y})$. We now show that $f_{\mathbf{y}}(\cdot; \mathbf{y}')$ is $\frac{1}{t}q(\mathbf{y}, \mathbf{y}') \cdot \|\mathbf{y}' - \mathbf{y}\|^2$ -smooth. Indeed, it is defined as the composition of the linear form $\boldsymbol{\theta} \mapsto \langle \boldsymbol{\theta}, \mathbf{y}' - \mathbf{y} \rangle$ and the function $g : \mathbb{R} \rightarrow \mathbb{R}$ given by:

$$g : x \mapsto \begin{cases} t \cdot q(\mathbf{y}', \mathbf{y}) \exp([x + \varphi(\mathbf{y}') - \varphi(\mathbf{y})] / t) & \text{if } x \leq t \log \frac{q(\mathbf{y}, \mathbf{y}')}{q(\mathbf{y}', \mathbf{y})} + \varphi(\mathbf{y}) - \varphi(\mathbf{y}'), \\ t \cdot q(\mathbf{y}, \mathbf{y}') \left([x + \varphi(\mathbf{y}') - \varphi(\mathbf{y})] / t + 1 - \log \frac{q(\mathbf{y}, \mathbf{y}')}{q(\mathbf{y}', \mathbf{y})}\right) & \text{if } x > t \log \frac{q(\mathbf{y}, \mathbf{y}')}{q(\mathbf{y}', \mathbf{y})} + \varphi(\mathbf{y}) - \varphi(\mathbf{y}'). \end{cases}$$

We begin by showing that g is $\frac{1}{t}q(\mathbf{y}, \mathbf{y}')$ -smooth. We have:

$$g' : x \mapsto \begin{cases} q(\mathbf{y}', \mathbf{y}) \exp([x + \varphi(\mathbf{y}') - \varphi(\mathbf{y})] / t) & \text{if } x \leq t \log \frac{q(\mathbf{y}, \mathbf{y}')}{q(\mathbf{y}', \mathbf{y})} + \varphi(\mathbf{y}) - \varphi(\mathbf{y}'), \\ q(\mathbf{y}, \mathbf{y}') & \text{if } x > t \log \frac{q(\mathbf{y}, \mathbf{y}')}{q(\mathbf{y}', \mathbf{y})} + \varphi(\mathbf{y}) - \varphi(\mathbf{y}'). \end{cases}$$

Thus, g' is continuous, and differentiable everywhere except in $x_0 := t \log \frac{q(\mathbf{y}, \mathbf{y}')}{q(\mathbf{y}', \mathbf{y})} + \varphi(\mathbf{y}) - \varphi(\mathbf{y}')$. Its derivative is given by:

$$g'' : x \mapsto \begin{cases} \frac{1}{t}q(\mathbf{y}', \mathbf{y}) \exp([x + \varphi(\mathbf{y}') - \varphi(\mathbf{y})] / t) & \text{if } x \leq t \log \frac{q(\mathbf{y}, \mathbf{y}')}{q(\mathbf{y}', \mathbf{y})} + \varphi(\mathbf{y}) - \varphi(\mathbf{y}'), \\ 0 & \text{if } x > t \log \frac{q(\mathbf{y}, \mathbf{y}')}{q(\mathbf{y}', \mathbf{y})} + \varphi(\mathbf{y}) - \varphi(\mathbf{y}'). \end{cases}$$

- For $x_1, x_2 \leq x_0$, we have:

$$\begin{aligned} |g'(x_1) - g'(x_2)| &\leq |x_1 - x_2| \sup_{x \in]-\infty, x_0[} |g''(x)| \\ &= |x_1 - x_2| \lim_{\substack{x \rightarrow x_0 \\ x < x_0}} |g''(x)| \\ &= \frac{1}{t}q(\mathbf{y}, \mathbf{y}') \cdot |x_1 - x_2|. \end{aligned}$$

- For $x_1, x_2 \geq x_0$, we trivially have $|g'(x_1) - g'(x_2)| = 0$.
- For $x_1 \leq x_0 \leq x_2$, we have:

$$\begin{aligned}
|g'(x_1) - g'(x_2)| &= |(g'(x_1) - g'(x_0)) - (g'(x_2) - g'(x_0))| \\
&\leq |g'(x_1) - g'(x_0)| + |g'(x_2) - g'(x_0)| \\
&\leq \frac{1}{t} q(\mathbf{y}, \mathbf{y}') \cdot |x_1 - x_0| \\
&\leq \frac{1}{t} q(\mathbf{y}, \mathbf{y}') \cdot |x_1 - x_2|.
\end{aligned}$$

Thus, we have:

$$\forall x_1, x_2 \in \mathbb{R}, |g'(x_1) - g'(x_2)| \leq \frac{1}{t} q(\mathbf{y}, \mathbf{y}') \cdot |x_1 - x_2|,$$

and g is $\frac{1}{t} q(\mathbf{y}, \mathbf{y}')$ -smooth. Nevertheless, we have $f_{\mathbf{y}}(\cdot, \mathbf{y}') = g(\langle \cdot, \mathbf{y}' - \mathbf{y} \rangle)$. Thus, we have, for $\theta_1, \theta_2 \in \mathbb{R}^d$:

$$\begin{aligned}
\|\nabla_{\theta} f_{\mathbf{y}}(\theta_1, \mathbf{y}') - \nabla_{\theta} f_{\mathbf{y}}(\theta_2, \mathbf{y}')\| &= \|g'(\langle \theta_1, \mathbf{y}' - \mathbf{y} \rangle)(\mathbf{y}' - \mathbf{y}) - g'(\langle \theta_2, \mathbf{y}' - \mathbf{y} \rangle)(\mathbf{y}' - \mathbf{y})\| \\
&= |g'(\langle \theta_1, \mathbf{y}' - \mathbf{y} \rangle) - g'(\langle \theta_2, \mathbf{y}' - \mathbf{y} \rangle)| \cdot \|\mathbf{y}' - \mathbf{y}\| \\
&\leq \frac{1}{t} q(\mathbf{y}, \mathbf{y}') \cdot |\langle \theta_1, \mathbf{y}' - \mathbf{y} \rangle - \langle \theta_2, \mathbf{y}' - \mathbf{y} \rangle| \cdot \|\mathbf{y}' - \mathbf{y}\| \\
&\leq \frac{1}{t} q(\mathbf{y}, \mathbf{y}') \cdot \|\mathbf{y}' - \mathbf{y}\|^2 \cdot \|\theta_1 - \theta_2\|,
\end{aligned}$$

and $f_{\mathbf{y}}(\cdot, \mathbf{y}')$ is $\frac{1}{t} q(\mathbf{y}, \mathbf{y}') \cdot \|\mathbf{y}' - \mathbf{y}\|^2$ -smooth. Thus, recalling that $F_{\mathbf{y}}$ is defined as

$$F_{\mathbf{y}} : \theta \mapsto \langle \theta, \mathbf{y} \rangle + \sum_{\mathbf{y}' \in \mathcal{N}(\mathbf{y})} f_{\mathbf{y}}(\theta; \mathbf{y}'),$$

we have that $F_{\mathbf{y}}$ is $\sum_{\mathbf{y}' \in \mathcal{N}(\mathbf{y})} \frac{1}{t} q(\mathbf{y}, \mathbf{y}') \cdot \|\mathbf{y}' - \mathbf{y}\|^2 = \mathbb{E}_{q(\mathbf{y}, \cdot)} \|Y - \mathbf{y}\|_2^2 / t$ -smooth. Finally, as $\Omega_{\mathbf{y}} := (F_{\mathbf{y}})^*$, Fenchel duality theory gives that $\Omega_{\mathbf{y}}$ is $t / \mathbb{E}_{q(\mathbf{y}, \cdot)} \|Y - \mathbf{y}\|_2^2$ -strongly convex.

$$\bullet \underline{\mathbb{E}_{p_{\theta, \mathbf{y}}^{(1)}} [Y] = \operatorname{argmax}_{\mu \in \operatorname{conv}(\mathcal{N}(\mathbf{y}) \cup \{\mathbf{y}\})} \{\langle \theta, \mu \rangle - \Omega_{\mathbf{y}}(\mu)\}}:$$

Noticing that g is continuous on $\mathbb{R}$, convex on $\left] -\infty, t \log \frac{q(\mathbf{y}, \mathbf{y}')}{q(\mathbf{y}', \mathbf{y})} + \varphi(\mathbf{y}) - \varphi(\mathbf{y}') \right]$ and on $\left[t \log \frac{q(\mathbf{y}, \mathbf{y}')}{q(\mathbf{y}', \mathbf{y})} + \varphi(\mathbf{y}) - \varphi(\mathbf{y}'), +\infty \right]$, and with matching derivatives on the junction:

$$g'(t) \xrightarrow[t < t \log \frac{q(\mathbf{y}, \mathbf{y}')}{q(\mathbf{y}', \mathbf{y})} + \varphi(\mathbf{y}) - \varphi(\mathbf{y}')]{t \rightarrow t \log \frac{q(\mathbf{y}, \mathbf{y}')}{q(\mathbf{y}', \mathbf{y})} + \varphi(\mathbf{y}) - \varphi(\mathbf{y}')} q(\mathbf{y}, \mathbf{y}'), \quad g'(t) \xrightarrow[t > t \log \frac{q(\mathbf{y}, \mathbf{y}')}{q(\mathbf{y}', \mathbf{y})} + \varphi(\mathbf{y}) - \varphi(\mathbf{y}')]{t \rightarrow t \log \frac{q(\mathbf{y}, \mathbf{y}')}{q(\mathbf{y}', \mathbf{y})} + \varphi(\mathbf{y}) - \varphi(\mathbf{y}')} q(\mathbf{y}, \mathbf{y}'),$$

gives that g is convex on $\mathbb{R}$. Thus, $f_{\mathbf{y}}(\cdot; \mathbf{y}')$ is convex on $\mathbb{R}^d$ by composition. Thus,

$$F_{\mathbf{y}} : \theta \mapsto \langle \theta, \mathbf{y} \rangle + \sum_{\mathbf{y}' \in \mathcal{N}(\mathbf{y})} f_{\mathbf{y}}(\theta; \mathbf{y}')$$

is closed proper convex as the sum of such functions. The Fenchel-Moreau theorem then gives that it is equal to its biconjugate. Thus, we have:

$$F_{\mathbf{y}}(\theta) = \sup_{\mu \in \mathbb{R}^d} \{\langle \theta, \mu \rangle - (F_{\mathbf{y}})^*(\mu)\} = \sup_{\mu \in \mathbb{R}^d} \{\langle \theta, \mu \rangle - \Omega_{\mathbf{y}}(\mu)\}.$$

Nonetheless, the gradient of $F_{\mathbf{y}}$ is given by:

$$\begin{aligned}
\nabla_{\theta} F_{\mathbf{y}}(\theta) &= \mathbf{y} + \sum_{\mathbf{y}' \in \mathcal{N}_{\mathbf{y}}^{-}(\theta)} q(\mathbf{y}', \mathbf{y}) \exp([\langle \theta, \mathbf{y}' - \mathbf{y} \rangle + \varphi(\mathbf{y}') - \varphi(\mathbf{y})] / t) \cdot (\mathbf{y}' - \mathbf{y}) + \sum_{\mathbf{y}' \in \mathcal{N}_{\mathbf{y}}^{+}(\theta)} q(\mathbf{y}, \mathbf{y}') \cdot (\mathbf{y}' - \mathbf{y}) \\
&= \mathbb{E}_{p_{\theta, \mathbf{y}}^{(1)}} [Y].
\end{aligned}$$

Thus, we have $\nabla F_{\mathbf{y}}(\mathbb{R}^d) \subset \text{conv}(\mathcal{N}(\mathbf{y}) \cup \{\mathbf{y}\})$, which gives:

$$\forall \boldsymbol{\theta} \in \mathbb{R}^d, \|\nabla F_{\mathbf{y}}(\boldsymbol{\theta})\| \leq R_{\mathcal{N}(\mathbf{y})} := \max_{\boldsymbol{\mu} \in \text{conv}(\mathcal{N}(\mathbf{y}) \cup \{\mathbf{y}\})} \|\boldsymbol{\mu}\|,$$

so that we have $\text{dom}(\Omega_{\mathbf{y}}) \subset B(\mathbf{0}, R_{\mathcal{N}(\mathbf{y})})$. Thus we can actually write:

$$F_{\mathbf{y}}(\boldsymbol{\theta}) = \sup_{\boldsymbol{\mu} \in B(\mathbf{0}, R_{\mathcal{N}(\mathbf{y})})} \{\langle \boldsymbol{\theta}, \boldsymbol{\mu} \rangle - \Omega_{\mathbf{y}}(\boldsymbol{\mu})\},$$

and now apply Danksin's theorem as $B(\mathbf{0}, R_{\mathcal{N}(\mathbf{y})})$ is compact, which further gives:

$$\partial F_{\mathbf{y}}(\boldsymbol{\theta}) = \underset{\boldsymbol{\mu} \in B(\mathbf{0}, R_{\mathcal{N}(\mathbf{y})})}{\text{argmax}} \{\langle \boldsymbol{\theta}, \boldsymbol{\mu} \rangle - \Omega_{\mathbf{y}}(\boldsymbol{\mu})\},$$

and the fact that $F_{\mathbf{y}}$ is differentiable gives that both sides are single-valued. Moreover, as $\nabla F_{\mathbf{y}}(\mathbb{R}^d) \subset \text{conv}(\mathcal{N}(\mathbf{y}) \cup \{\mathbf{y}\})$, we know that the right hand side is maximized in $\text{conv}(\mathcal{N}(\mathbf{y}) \cup \{\mathbf{y}\})$, and we can actually write:

$$\mathbb{E}_{\mathbf{p}_{\boldsymbol{\theta}, \mathbf{y}}^{(1)}}[Y] = \nabla F_{\mathbf{y}}(\boldsymbol{\theta}) = \underset{\boldsymbol{\mu} \in \text{conv}(\mathcal{N}(\mathbf{y}) \cup \{\mathbf{y}\})}{\text{argmax}} \{\langle \boldsymbol{\theta}, \boldsymbol{\mu} \rangle - \Omega_{\mathbf{y}}(\boldsymbol{\mu})\}.$$

- Smoothness of $L_{\Omega_{\mathbf{y}}}(\cdot; \mathbf{y})$ and expression of its gradient:

Based on the above, we have:

$$L_{\Omega_{\mathbf{y}}}(\boldsymbol{\theta}; \mathbf{y}) = F_{\mathbf{y}}(\boldsymbol{\theta}) + \Omega_{\mathbf{y}}(\mathbf{y}) - \langle \boldsymbol{\theta}, \mathbf{y} \rangle.$$

Thus, the $\mathbb{E}_{q(\mathbf{y}, \cdot)}\|Y - \mathbf{y}\|_2^2/t$ -smoothness of $L_{\Omega_{\mathbf{y}}}(\cdot; \mathbf{y})$ follows directly from the previously established $\mathbb{E}_{q(\mathbf{y}, \cdot)}\|Y - \mathbf{y}\|_2^2/t$ -smoothness of $F_{\mathbf{y}}$. Similarly, the expression of $\nabla_{\boldsymbol{\theta}} L_{\Omega_{\mathbf{y}}}(\boldsymbol{\theta}; \mathbf{y})$ follows from the previously established expression of $\nabla_{\boldsymbol{\theta}} F_{\mathbf{y}}(\boldsymbol{\theta})$, and we have:

$$\nabla_{\boldsymbol{\theta}} L_{\Omega_{\mathbf{y}}}(\boldsymbol{\theta}; \mathbf{y}) = \nabla_{\boldsymbol{\theta}} F_{\mathbf{y}}(\boldsymbol{\theta}) - \mathbf{y} = \mathbb{E}_{\mathbf{p}_{\boldsymbol{\theta}, \mathbf{y}}^{(1)}}[Y] - \mathbf{y}.$$

□

Proposition 6. In the unsupervised setting, given a dataset $(\mathbf{y}_i)_{i=1}^N$, the distribution of the first iterate of the Markov chain with transition kernel defined in Eq. (3) and initialized by $\mathbf{y}^{(0)} = \mathbf{y}_i$, with $i \sim \mathcal{U}([1, N])$, is given by:

$$\begin{aligned} (\mathbf{p}_{\boldsymbol{\theta}, \tilde{Y}_N}^{(1)})(\mathbf{y}) &= \sum_{\mathbf{y}' \in \mathcal{Y}} \left(\sum_{i=1}^N \mathbf{1}_{\{\mathbf{y}_i = \mathbf{y}'\}} \cdot \frac{1}{N} \right) P_{\boldsymbol{\theta}, t}(\mathbf{y}', \mathbf{y}) \\ &= \sum_{\mathbf{y}' \in \mathcal{Y}} \left(\sum_{i=1}^N \mathbf{1}_{\{\mathbf{y}_i = \mathbf{y}'\}} \cdot \frac{1}{N} \right) \mathbf{p}_{\boldsymbol{\theta}, \mathbf{y}'}^{(1)}(\mathbf{y}) \\ &= \frac{1}{N} \sum_{i=1}^N \mathbf{p}_{\boldsymbol{\theta}, \mathbf{y}_i}^{(1)}(\mathbf{y}). \end{aligned}$$

Thus, keeping the same notations as in the previous proof, previous calculations give:

$$\begin{aligned} \mathbb{E}_{\mathbf{p}_{\boldsymbol{\theta}, \tilde{Y}_N}^{(1)}}[Y] &= \frac{1}{N} \sum_{i=1}^N \mathbb{E}_{\mathbf{p}_{\boldsymbol{\theta}, \mathbf{y}_i}^{(1)}}[Y] \\ &= \frac{1}{N} \sum_{i=1}^N \nabla_{\boldsymbol{\theta}} F_{\mathbf{y}_i}(\boldsymbol{\theta}) \\ &= \nabla_{\boldsymbol{\theta}} \left(\frac{1}{N} \sum_{i=1}^N F_{\mathbf{y}_i} \right) (\boldsymbol{\theta}). \end{aligned}$$

Let $F_{\bar{Y}_N} := \frac{1}{N} \sum_{i=1}^N F_{\mathbf{y}_i}$. Then, the exact same arguments as in the supervised case hold, and the results of Proposition 6 are obtained by replacing $F_{\mathbf{y}}$ by $F_{\bar{Y}_N}$ in the proof of Proposition 3, and noticing that the previously shown $\mathbb{E}_{q(\mathbf{y}_i, \cdot)} \|Y - \mathbf{y}_i\|_2^2/t$ -smoothness of $F_{\mathbf{y}_i}$ gives that $F_{\bar{Y}_N}$ is $\frac{1}{N} \sum_{i=1}^N \mathbb{E}_{q(\mathbf{y}_i, \cdot)} \|Y - \mathbf{y}_i\|_2^2/t$ -smooth. Similar arguments also hold for the regularized optimization formulation, by noting that this time we have $\nabla F_{\bar{Y}_N}(\mathbb{R}^d) \subset \text{conv} \left(\bigcup_{i=1}^N \{\mathcal{N}(\mathbf{y}_i) \cup \{\mathbf{y}_i\}\} \right)$. $\square$